

BTF Trace Viewer

Version 1.4.0 — Desktop (Python) and Web

Interactive viewer for FreeRTOS context-switch traces in **Best Trace Format** (.btf). Open plain or compressed traces, inspect scheduling on a timeline, measure with cursors, review statistics, and export reports or screenshots.

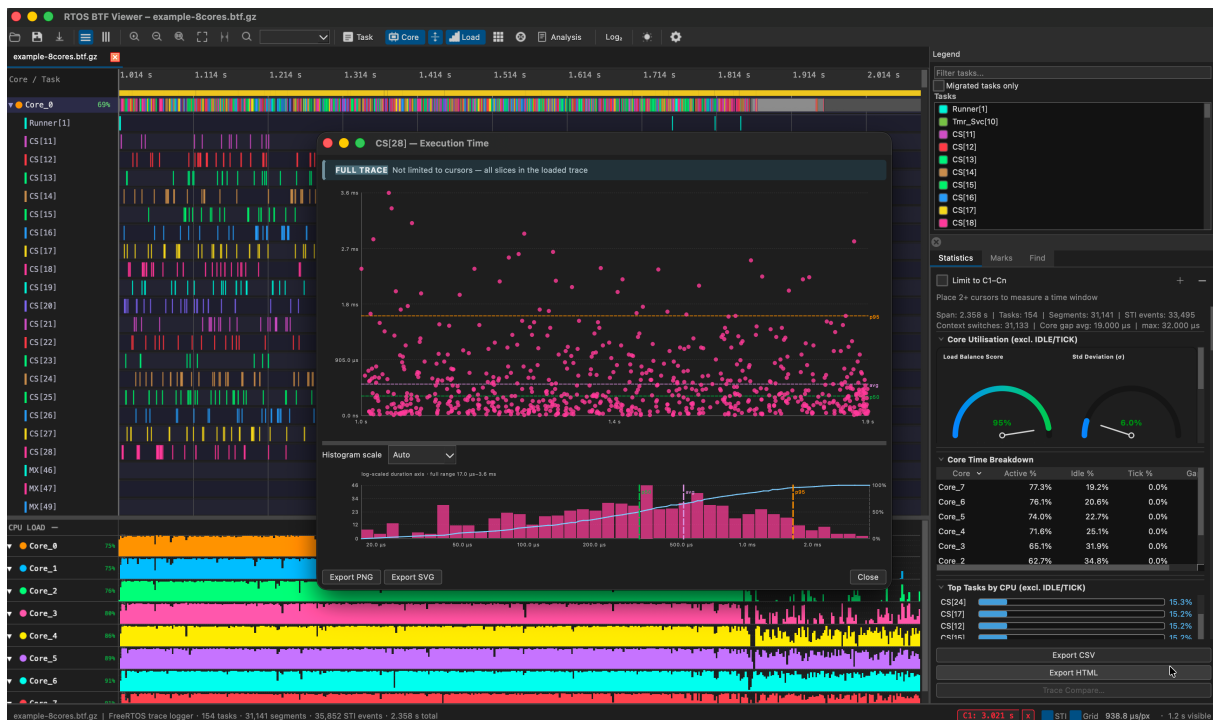


Figure 1: BTF Viewer screenshot

[Try the live demo](#)

Features

- **Timeline visualisation** — task and core views, horizontal or vertical layout, smooth zoom and pan
- **Multi-trace sessions** — open several traces in tabs; compare builds side by side

-
- **Measurement tools** — cursors, bookmarks, annotations, find & jump
 - **Statistics & analysis** — utilisation, latency, migrations, sync/mutex metrics, and severity-tagged Analysis Findings
 - **AI Assistant** — ask diagnostic questions over Analysis Findings (local or cloud models)
 - **Export** — annotated snapshots (PNG/SVG), CSV/HTML reports, Perfetto traces
 - **Desktop CLI** — headless reports and images for scripts and CI
 - **Desktop and Web** — the same analysis workflow in a desktop app or the browser

Metric definitions and formulas are in Statistics & metrics. Step-by-step diagnosis playbooks: **WORKFLOWS.md**.

Contents

Section

Quick start	Install and open a trace
Using the viewer	Navigate, measure, and mark
Analysis	Findings, problem map, compare, AI
Statistics & metrics	Metric meaning, formulas, charts, how to diagnose
Export	Snapshots, reports, Perfetto, CLI

Section

Settings	Preferences and defaults
Keyboard & mouse	Shortcuts reference
Further reading	Workflows and developer notes

Quick start

Desktop

Requirements: Python 3.8+ and PySide6 $\geq$ 6.4.

```
cd BTFViewer
pip install -r requirements.txt
python builds/btf_viewer.py [trace.btf]
```

With no file argument, the previous session is restored. You can also use **File** → **Open**, drag-and-drop, or **File** → **Open Recent**.

Web

Open the standalone build in any modern browser (no server required for typical use):

```
open BTFViewer/builds/btf_viewer.html # or double-click the file
```

Or use the hosted demo. To rebuild from source:

```
cd BTFViewer && make web # → builds/btf_viewer.html
```

Supported files

Format	Notes
.btf	Best Trace Format
.btf.gz / .gz, .btf.bz2 / .bz2	Compressed single traces
.btf.zip / .zip	One or more .btf members — each opens in its own tab

Sample traces are under `tracedata/` (for example `example-2cores.btf.gz`).

Using the viewer

Desktop and Web share the same concepts. Where behaviour differs, it is noted below.

View modes and orientation

Control	Purpose
Task View	One row (or column) per task
Core View	One expandable row per core; expand a core or use Expand / Collapse All
Horizontal / Vertical	Time left→right or top→bottom

Switch from the toolbar or **View** menu. The last orientation is remembered.

Zoom and pan

- **Wheel** — pan (add **Ctrl** to zoom; **Shift** swaps axes)
- **Pinch** (macOS) — zoom
- **Middle-drag** — select a time range to zoom
- **Fit** (Ctrl+0 / F) — show the full trace
- **1:1** — reset to the configured zoom density
- **Zoom to cursor range** (Ctrl+R) — fit between the first and last cursor

Labels, legend, and highlight

- Drag the label-column edge to resize; double-click the edge to auto-fit.
- The **Legend** lists tasks with colours and filters (including migrated tasks).
- Click or hover a task label or legend row to highlight that task on the timeline.
- Hover a segment for duration, core, and neighbouring activity.

CPU Load

Toggle **Load** on the toolbar for a utilisation chart under the timeline. Drag the divider to resize. With two or more cursors, the chart can show a cursor-range average.

With a task **lock-highlighted**, Task View shows that task's utilisation **per core** in the load strip — useful when investigating migrations (see Highlight a migrating task).

Cursors

Place up to **4-8** cursors (default 4; set in Settings).

Action	How
Place	Click the timeline, or press C at the viewport centre
Move	Drag the cursor line
Remove	Click near a cursor, or use the context menu
Clear all	Shift+C or Shift+right-click

Action	How
Snap to boundary	Shift+click

With **two or more** cursors, the status bar shows a short range summary, and Statistics can limit metrics to **C1-Cn (Limit to cursor range)**.

Marks and find

Tool	Purpose
Bookmarks	Named timestamps (Ctrl+B)
Annotations	Free-text notes at a time (Ctrl+Shift+B / A)
Find	Search tasks, migrations, STI events, intervals, and more (Ctrl+F; F3 / Shift+F3 to step)

Right-click the timeline for cursor, bookmark, and annotation actions. Marks and cursors are restored with the session.

Multi-tab traces

Open several files (or a multi-BTF zip) as tabs. Each tab keeps its own zoom, cursors, marks, and filters. Cycle with Ctrl+Tab / Ctrl+Shift+Tab; close with Ctrl+W.

Tag and STI markers

Software-trace items appear as markers (and optional tag channels). Toggle STI rows with I or Settings. Expand tag channels for waveforms where available.

Analysis

Start with toolbar **Analysis** for a severity-tagged triage of the current Statistics scope, then open the named Statistics sections. For a top-down inspection order and worked examples, see **WORKFLOWS.md**.

Analysis Findings

Toolbar **Analysis** summarises likely issues for the current scope (load imbalance, WCET/CPU hotspots, blocking, priority inversion, core thrashing, deadline breaches, tick health, sync/mutex bounces, and similar). Use **Save as text...** for a copy; the same card appears in **Export HTML**.

How to act on a finding

1. Note the severity and the Statistics section it names.
2. Open that section, sort by Max / Rate / Bounce as relevant.
3. Click **Min** / **Max**, a chart point, or a heatmap cell to jump the timeline.
4. Place cursors around the phase of interest and enable **Limit to cursor range**.
5. Optionally ask the **AI** tab to walk the same findings (WORKFLOWS.md §7).

How to find problems (quick map)

Symptom	Start here	Then check
Unknown — triage first	Toolbar Analysis	Named Statistics sections
Tick jitter / tickless	Trace Health (TICK)	Scope a busy window; Execution Max
SMP uneven load	Core Utilisation (Score / σ)	Concurrent Core Active; Migrations
High scheduler cost	Kernel Switch Overhead	Core Time Breakdown (Gap %)
Task too slow on CPU	Execution Time (Max / p95)	Preemption Chain, Mutex
Task waits too long	Blocking Time	Preemption Chain, Mutex

Symptom	Start here	Then check
Ready→run delay	Dispatch Latency	Blocking, Preemption (needs STI resume)
Priority inversion	Priority Inheritance	Mutex pairing, Blocking
Core thrashing	Core Migrations (Rate, Ping)	Heatmap / Chord; Mutex Bounces
Lock / queue issues	Mutex / Semaphore / Queue	Blocking, Migrations
Before/after change	Trace Compare	Same cursor phases on both tabs

Trace Compare

With **two or more** tabs open, **Trace Compare...** diffs summary, top tasks, utilisation, migrations, execution, blocking, inter-arrival, preemption, and sync. Optionally limit each side to its own cursor range. Export CSV/HTML from the dialog. See also Core migration analysis below.

AI Assistant

The **AI** tab answers questions using **Analysis Findings** (or Trace Compare tables for that template)—not the raw BTF stream.

Any OpenAI-compatible endpoint works, including Ollama’s own (<http://localhost:11434/v1>).

1. **Settings → AI** — pick a preset (**Ollama**, **OpenAI**, **Google Gemini**, or **Custom**), adjust base URL / model / API key, then **Test connection**. Each preset keeps its own settings, so switching back and forth never loses a key.
2. **Import...** loads an endpoint from a JSON file — see [examples/ai](#) for `gemini.json`, `openai.json`, `deepseek.json`, `grok.json`, and the field reference. Imported values fill the form; review and confirm to save.
3. Use a template or ask freely; click `jump:TIME` links to seek the timeline.
4. Set reply language in Settings or **Language...** on the AI bar.
5. Right-click the reply area to copy the conversation or **Save As...** it (Markdown, plain text, or HTML).

If this happens	Try
Web: Failed to fetch / CORS	The server is usually fine — the page just is not allowed to call it. Prefer <code>npm run dev / make preview</code> (both proxy Ollama), or see Opening the web app from <code>file://</code>
401 / 403	Check API key (also read from <code>OPENAI_API_KEY / GEMINI_API_KEY / OLLAMA_API_KEY</code> ; local Ollama needs none)
Model not found	Test connection lists the models the endpoint serves — pick one of those, or <code>ollama pull</code> it

Opening the web app from `file://`

A page opened straight from disk sends `Origin: null`, which Ollama rejects with 403 — the browser then reports only `Failed to fetch`. Serving the app over `http` avoids this entirely (`npm run dev / make preview` proxy Ollama for you), and the Desktop app is not affected at all.

To keep using `file://`, allow every origin on the Ollama side:

```
# Server started from a terminal
OLLAMA_ORIGINS="*" ollama serve
```

```
# macOS menu-bar app (Ollama.app) – a shell variable does not reach it
launchctl setenv OLLAMA_ORIGINS "*" # then quit Ollama and reopen it
```

Verify the change took effect; expect 200 and an Access-Control-Allow-Origin header:

```
curl -s -D - -o /dev/null -H "Origin: null" http://localhost:11434/v1/models
↪ \
  | grep -iE "^HTTP|access-control-allow-origin"
```

If a file:// page is still refused, list the null origin explicitly with `OLLAMA_ORIGINS="*,null"`. Note that `*` lets **any** page you visit reach your local models; undo it with `launchctl unsetenv OLLAMA_ORIGINS` when done.

Ask-order tips: WORKFLOWS.md §7.

Statistics & metrics

Shared by the Desktop **Statistics** tab and the Web **Statistics** panel.

The **Statistics** tab in the right-side panel (Desktop dock + Web tab) shows per-core CPU utilisation, top tasks, scheduling summary, trace health, and collapsible metric tables. Toggle visibility from **Settings** → **Display** → **Statistics panel**. For a quick triage of the same scope, use toolbar **Analysis** (see Analysis Findings) — findings are **not** listed as a panel section.

At the top, **Limit to cursor range (C1-Cn)** restricts all statistics (and Analysis Findings) to the time window from the first placed cursor through the last (requires 2+ cursors). Section titles show **(cursor range)** when scoped. Clearing all cursors returns to full-trace statistics immediately.

Panel header controls (next to the cursor-range checkbox):

Control	Action
+	Expand all sections
–	Collapse all sections (pinned sections stay open)

Control	Action
Reset-order (circular arrow)	Restore the built-in section sequence; disabled / grayed out when order is already default

Section header controls (each metric table):

Control	Action
Section title / chevron	Expand or collapse that section (no-op while pinned)
≡ grip	Drag onto another section to reorder (destination shows a dashed highlight)
Pin (thumbtack)	Keep the section expanded; survives Collapse all and heavy-trace deferral

Pins, section order, and table heights persist across launches. Drag the splitter between the timeline and CPU load to resize that pane.

Default section sequence (customize via drag-reorder):

- **Summary** — span, task/segment/STI counts (scoped when the checkbox is on)
- **Scheduling summary** — context-switch count and average/max core gap between consecutive slices on each core
- **Core utilisation** — percentage of active (non-IDLE, non-TICK) CPU time per core; side-by-side **Load Balance Score** and σ gauges at the top (Gini; red / amber / green zones) (collapsible)
- **Trace health (TICK)** — tick period regularity, **tick / tickless mode detection** (coefficient of variation of tick intervals), large gaps, missed-tick estimate, and **Tick Distribution...** chart button (bar-chart icon beside the mode badge when ≥ 2 ticks in scope) (collapsible)
- **Core Time Breakdown** — per-core time budget split into **Active** (non-IDLE, non-TICK tasks), **Idle** (IDLE task), **Tick** (TICK handler), and **Gap** (unaccounted span between segments — scheduler latency / ISR overhead) expressed as percentages (collapsible)

-
- **Concurrent Core Active Distribution** — fraction of the scoped window spent with N cores concurrently running non-IDLE/non-TICK work (0...N); complements Load Balance Score with a temporal parallelism view. Click a row for the interval-duration distribution (collapsible)
 - **Kernel Switch Overhead** — per-core context-switch cost $t_{\text{resume},B} - t_{\text{preempt},A}$ from consecutive core_segs gaps: switches, min/avg/max, total overhead, % of core span. Click a row for the switch-gap distribution (collapsible)
 - **Top tasks by CPU** — ranked list of worker tasks by total CPU time consumed (collapsible)
 - **Core Migrations** — per-task migration count, **migration rate** (normalized per second of active time and per scheduler tick), **average core dwell time**, core count, primary core (% time), ping-pong count, STI events near migrations, and average off-CPU gap after migration vs other gaps; click a row for a distribution chart with **Dwell / Rate / Gap** tabs (collapsible)
 - **Core-Pair Migration Summary** — per directed core-pair (e.g. Core_0 → Core_1): total migration count, lock-bounce count and percentage, and average off-CPU gap after migration; **click a row** for Gap / Rate distribution charts (bounce samples in orange) with **Open Heatmap / Open Chord** deep-links (collapsible; shown only on multi-core traces with migrations)
 - **Core Affinity** — per-task affinity mask history from affinity_set STI (vTaskCoreAffinitySet) vs. observed execution cores; **Violations** lists cores used while outside the mask then in effect (slices before the first set are unrestricted; mask changes are shown as 0x1 → 0x8) (collapsible; shown only when those STI events are present)
 - **Task Lifecycle** — per-task summary of create, delete, suspend, and resume events recorded on the task STI channel: created/deleted timestamps, suspend/resume counts, alive span (create→delete), total lifecycle event count, and **Runs** — the number of times the task was actually dispatched onto a core (context-switch-in / segment count). Runs is a scheduler-level metric and is normally much larger than Susp/Res, which only counts explicit vTaskSuspend()/vTaskResume() API calls — a task can run (and be preempted/resumed by the scheduler) many times without ever being suspended. In example-8cores.bt.gz, test 9 creates **SR0-SR3** (pinned across cores) with overlapping suspend-while-blocked and suspend-while-running rounds — use this section to confirm Susp/Res counts match the STI pairs (collapsible; shown only when the trace contains task lifecycle STI events)
 - **Deadlines / CPU budget** — configurable per-task execution deadline
-

-
- (**nanoseconds**, converted to the trace `#timeScale` before compare) and global CPU budget threshold (%); shows **Slice over deadline** (top 20 by duration; click a row to jump + annotate) and **CPU budget exceeded** (click a row to highlight the task); click column headers to sort; click **Settings** → **Display** in the section to open Analysis thresholds (Ctrl+,) — always visible, with a configure prompt when no thresholds are set (collapsible)
- **Execution Time Per Slice** — per-task min/avg/max/jitter/ σ /p95, run count, and CPU%; click a row for a scatter + histogram popup; click **Min** / **Max** to jump and annotate the BCET / WCET slice
 - **Blocking Time** — off-CPU gap between consecutive activations of the same task (not end-to-end response time); min/avg/max/jitter/ σ /p95; click a row for a distribution chart; click **Min** / **Max** to jump and annotate the shortest / longest off-CPU gap (collapsible)
 - **Dispatch / Scheduling Latency** — ready→run delay using STI resume Name[id] (vTaskResume) or create→first-run as tready, and the next switch-in as tresume; sync-object wakes are not attributed (BTF notes lack the woken task id). Click a row for the distribution plot; click **Min** / **Max** to jump to the extreme dispatch (collapsible)
 - **Inter-Arrival Time** — the same variability statistics for gaps between task activation starts; click a row for a distribution chart; click **Min** / **Max** to jump and annotate the shortest / longest inter-arrival gap (collapsible)
 - **Preemption Chain Analysis** — for each victim/preemptor pair: count, total/average/max preemption overlap; click a row for a distribution chart; click a scatter point to jump and add an annotation at the preemptor segment (collapsible)
 - **Priority Inheritance** — per-task base/peak priority, boost episodes, boosted time, and pattern (mutex inherit / L/M/H / boost only); click a row for a duration plot; click a scatter point to zoom, highlight, and annotate the episode (collapsible; shown only when the trace contains priority-inheritance STI events)
 - **Mutex / Semaphore pairing** — per-object hold count, issue count, **core bounce count**, average hold, and status; **Pairing issues** sub-table lists orphan gives, cross-task gives, unmatched takes, teardown warnings, and **CORE_MIGRATION_WHILE_HELD** warnings (lock that crossed core boundaries while held); click an issue row to zoom, jump, and annotate (collapsible; shown only when the trace contains sync-object STI events)
 - **Queue** — per-queue send/recv pairing by object pointer: hold count, issue count, average hold, and status (collapsible; shown only when the trace contains queue STI events)
-

-
- **Interval Analysis** — per interval id: count, min/avg/max/p95 duration of paired start→stop spans; pairing uses tid in the note when present; click a row for a duration plot; click a scatter point to jump and add an annotation at the interval start (collapsible)
 - **Tag Analysis** — per tag0_event...tag7_event STI channel: sample count, min/avg/max/p95 of the tag value; click a row to open a scatter + histogram plot (collapsible; shown only when the trace contains tag STI samples)

Export CSV / Export HTML respect the current cursor scope. **Export CSV** includes summary tables for every statistics section (including **Deadlines / CPU budget** when thresholds are configured), a **Core Affinity Violations** sub-table listing every mutex that crossed core boundaries, plus **Load Balance Score**, σ , and Gini coefficient under Core Utilisation. **Export HTML** adds the same summaries plus an **Analysis Findings** card near the top (same heuristics as toolbar **Analysis** — load imbalance, WCET/CPU hotspots, blocking, L/M/H priority inversion, core thrashing / hot pairs, deadline breaches, tick health, sync/mutex bounces), embeds the **Load Balance Score** gauge as a self-contained SVG `<img>` (data URI) under Core Utilisation, and detail sub-tables for **Priority Inheritance** (boost episodes), **Mutex / Semaphore** (pairing issues with bounce warnings, and hold episodes with take/give core columns), and **Interval Analysis** (individual instances). **Trace Compare...** diffs Summary (incl. load balance + tick health), Top Tasks, Core Util, Migrations, Execution, Blocking, Inter-Arrival, Preemption, and Sync between two open tabs; enable **Limit to each tab's cursor range** to scope each side to its own C1-Cn window. Open metrics charts update live when cursors move or scope is toggled; each trace tab remembers its own open chart when you switch tabs.

Full column definitions, chart axis meanings, and example plots: Statistics metric tables.

Statistics metric tables

The Statistics panel (Desktop **Statistics** tab + Web **Statistics** tab) organises metrics into collapsible sections. Tables are **sortable** — click a column header to sort ascending/descending. **Export CSV** and **Export HTML** at the panel footer honour the current cursor scope and include every section's summary table. **Export HTML** starts with an **Analysis Findings** card (same content as toolbar **Analysis** — load balance, WCET, blocking, thrashing, deadlines, tick health, and sync) and embeds the **Load Balance Score** gauge as an SVG image under Core Utilisation; use the

toolbar dialog for interactive triage and **Save as text....** HTML also adds detail sub-tables under Priority Inheritance, Mutex / Semaphore, and Interval Analysis (longest instances / hold episodes first, capped at 150–200 rows per sub-table).

How to use the panel

1. Open a trace (e.g. `tracedata/example-4cores.btf.gz` for a 4-core SMP workload, or `tracedata/example-2cores.btf.gz` for a smaller 2-core demo).
2. Optionally click toolbar **Analysis** for a severity-tagged triage of the current scope.
3. Expand the sections you care about (or use the **+** / **–** icons at the top to expand/collapse all). Pin frequently used sections so they stay open.
4. Optionally drag **≡** grips to reorder sections for your workflow; use the reset-order icon when you want the built-in sequence back.
5. Optionally place **2+ cursors** and enable **Limit to cursor range (C1-Cn)** to restrict every metric (and Analysis Findings) to a time window.
6. Click a **table row** to open a distribution chart (where supported), click **Min / Max** to jump and add an annotation at an extreme slice on the timeline, click a **Mutex / Semaphore** issue row to zoom, jump, and annotate at that STI event, or in **Deadlines / CPU budget** click a slice row to annotate / a CPU-budget row to highlight the task (or use the **Settings → Display** link to edit thresholds).
7. Use **Trace Compare...** when two traces are open to diff summary and migration stats.

Example plots below use `tracedata/example-8cores.btf.gz`. Worked diagnosis steps for that sample: WORKFLOWS.md §3.

Summary, scheduling, and core utilisation These sections open the Statistics panel in the **default** order (Summary and Scheduling summary are always first; **Core utilisation** is the first pinnable section). Later subsections below follow the same catalogue order as the app (cores → health → core_breakdown → concurrency → switch_overhead → tasks → ... → tags). Drag-reorder may place them elsewhere.

Summary — scope-wide counts: trace span, tasks, segments, STI events. Span is $t_{\max} - t_{\min}$ in the active scope (full trace or cursor range).

Scheduling summary — per core, **context switches** count slice boundaries and **core gap** is idle time between consecutive slices on that core:

$$g_{\text{core}} = t_{\text{start},k+1} - t_{\text{end},k}$$

Large **max core gap** on a core that should be busy suggests starvation, tickless idle, or a single long-running task blocking others.

Core utilisation — per core, share of non-IDLE, non-TICK active time in scope:

$$U_{\text{core}} = \frac{T_{\text{active,core}}}{T_{\text{scope}}} \times 100$$

When two or more cores are present, two gauges are shown side by side at the top of the section — **Load Balance Score** and **Std Deviation (σ)**:

$$\text{Score} = 100 \% \times (1 - G)$$

where G is the Gini coefficient of $\{U_{\text{core}}\}$.

σ is the population standard deviation of $\{U_{\text{core}}\}$. The Score needle uses a 0–100 % scale (100 = perfect balance, 0 = single-core overload); the σ needle uses a 0–60 % scale with the warn threshold at mid-scale. Zones match toolbar **Analysis**:

Zone	Condition	UI
Red	Score < 70 %	Unbalanced chip + alert (red zone); Score gauge red
Amber	Score $\geq$ 70 % and $\sigma >$ 30 %	$\sigma >$ 30% chip; σ gauge amber (red if $\sigma >$ 50 %)
OK (green)	Score $\geq$ 70 % and $\sigma \leq$ 30 %	Green needles

Toolbar **Analysis** also warns when Score < 70 % or $\sigma >$ 30 %, and only describes cores as “reasonably balanced” when Score $\geq$ 85 % and $\sigma \leq$ 30 %. **Export HTML** includes the gauges; **Export CSV** includes the score, σ , and G values.

What it tells you: Imbalanced utilisation across cores may indicate poor affinity, lock pinning, or workload placement issues — cross-check with **Core Migrations**, the migration heatmap, and toolbar **Analysis**.

Trace Health (TICK) Uses STI **TICK** timestamps to estimate scheduler tick regularity and detect whether the trace was recorded with the standard periodic tick or FreeRTOS **tickless idle** (configUSE_TICKLESS_IDLE).

Formula — for consecutive TICK events at times t_n :

$$\Delta_n = t_n - t_{n-1}, \quad \mu = \text{mean}(\Delta_n), \quad \sigma = \text{stdev}(\Delta_n), \quad \text{CV} = \frac{\sigma}{\mu}$$

Missed ticks (est.) counts large gaps where $\Delta_n \gg \mu$ (roughly $[\Delta_n / \mu] - 1$).

What it tells you: In **TICK** mode ($\text{CV} \leq 5\%$), intervals cluster at one nominal period — the scheduler clock is steady. **TICKLESS** mode ($\text{CV} > 5\%$) widens the distribution because idle periods suppress the tick interrupt; tall spikes in the scatter plot are multi-tick sleeps, not necessarily overload. Large **Max gap** with **good** status may still be worth inspecting for long critical sections or trace dropouts.

Field	Meaning
Status	good / warning / critical based on gap threshold
Mode badge	TICK (blue) or TICKLESS (amber) — detected automatically from the coefficient of variation ($\text{CV} = \sigma/\mu$) of consecutive tick intervals. $\text{CV} > 5\%$ is classified as tickless. Hover the badge to see the exact CV value
Ticks	TICK event count in scope
Avg period / Max gap	Observed tick spacing
Missed ticks (est.)	Rough count of skipped ticks from large gaps

In **tick mode** the timer interrupt fires at a constant rate and tick intervals form a tight cluster. In **tickless mode** the scheduler suppresses the tick interrupt during

idle periods to save power, so consecutive intervals span one or many nominal tick periods — the distribution widens significantly.

When tick intervals can be charted, a **Tick Distribution...** button (bar-chart icon, theme-aware amber/orange styling) appears beside the **TICK / TICKLESS** mode badge when ≥ 2 ticks are in scope (Desktop + Web). Clicking it opens the standard scatter + histogram popup showing: - **Scatter plot** — each tick interval over trace time; long idle periods appear as tall spikes. - **Histogram** — interval distribution with adaptive scaling and a CDF overlay (auto may choose p5-p95 or log duration when tickless idle stretches the range); clearly multi-modal in tickless mode (one sharp peak at $1 \times$ period, another at $2\times$, $3\times$, etc.). The CDF helps quantify what fraction of tick intervals are a single period vs two or more.

Click any scatter point to jump to that tick time, add an **annotation**, and open the **Marks** tab with the annotation selected (same behaviour as other metric distribution charts).

Tick Distribution in `example-8cores.btf.gz` (~ 2496 TICK events, $CV \approx 35.9\%$ → **TICKLESS**):

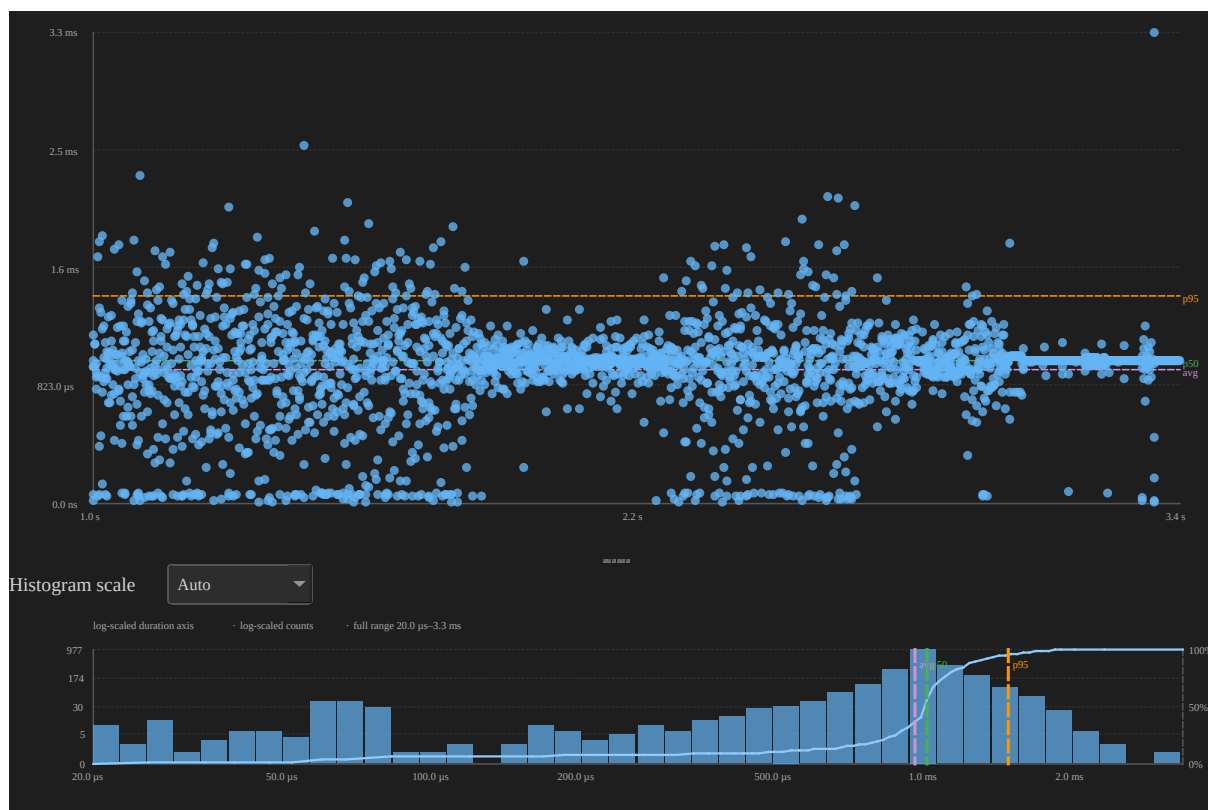


Figure 2: Tick interval distribution chart — scatter and histogram of consecutive TICK gaps in example-8cores.btf.gz

The histogram’s multiple peaks (1×, 2×, 3× the nominal period) confirm tickless idle: most gaps are a single tick, but idle stretches skip several nominal periods before the next TICK fires.

Large gaps may indicate CPU overload, long critical sections, tickless idle, or tracing gaps — not necessarily a FreeRTOS configuration error.

Core Time Breakdown Per-core time budget for the scoped window, split into four mutually exclusive buckets (percentages of the core span):

Bucket	Meaning
Active	Non-IDLE, non-TICK task time
Idle	IDLE task time

Bucket	Meaning
Tick	TICK handler time
Gap	Unaccounted span between consecutive core_segs (scheduler latency / ISR overhead / tracing gaps)

What it tells you: High **Gap** % on a busy core often correlates with elevated **Kernel Switch Overhead** or long ISR/critical sections. High **Idle** % with low **Active** % on some cores and overload on others points to affinity or placement imbalance — cross-check **Core utilisation** and **Core Migrations**. Click a core row (Desktop) to focus that core in **Core View**.

Concurrent Core Active Distribution Temporal parallelism: how much of the scoped window is spent with exactly N cores concurrently running non-IDLE/non-TICK work ($N = 0 \dots \text{number of cores}$).

Formula — let $\text{isActive}(c,t)$ be 1 when core c runs a non-IDLE/non-TICK task at time t :

$$N_{\text{active}}(t) = \sum_c \text{isActive}(c, t)$$

The table aggregates dwell time at each level N .

Column	Meaning
Active Cores	Concurrency level N
Duration	Total time spent with $N_{\text{active}} = N$
% of Span	Share of the scoped window

Distribution chart — click any row: scatter of interval start vs dwell duration while that many cores were active; histogram of those dwells. Complements **Load Balance Score** (which is utilisation balance, not simultaneous activity).

Headless snapshot (example-8cores.btf.gz, $N = 4$ active cores):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o stats-concurrency-4.svg --view plot --metric concurrency
↪ --active-cores 4
```

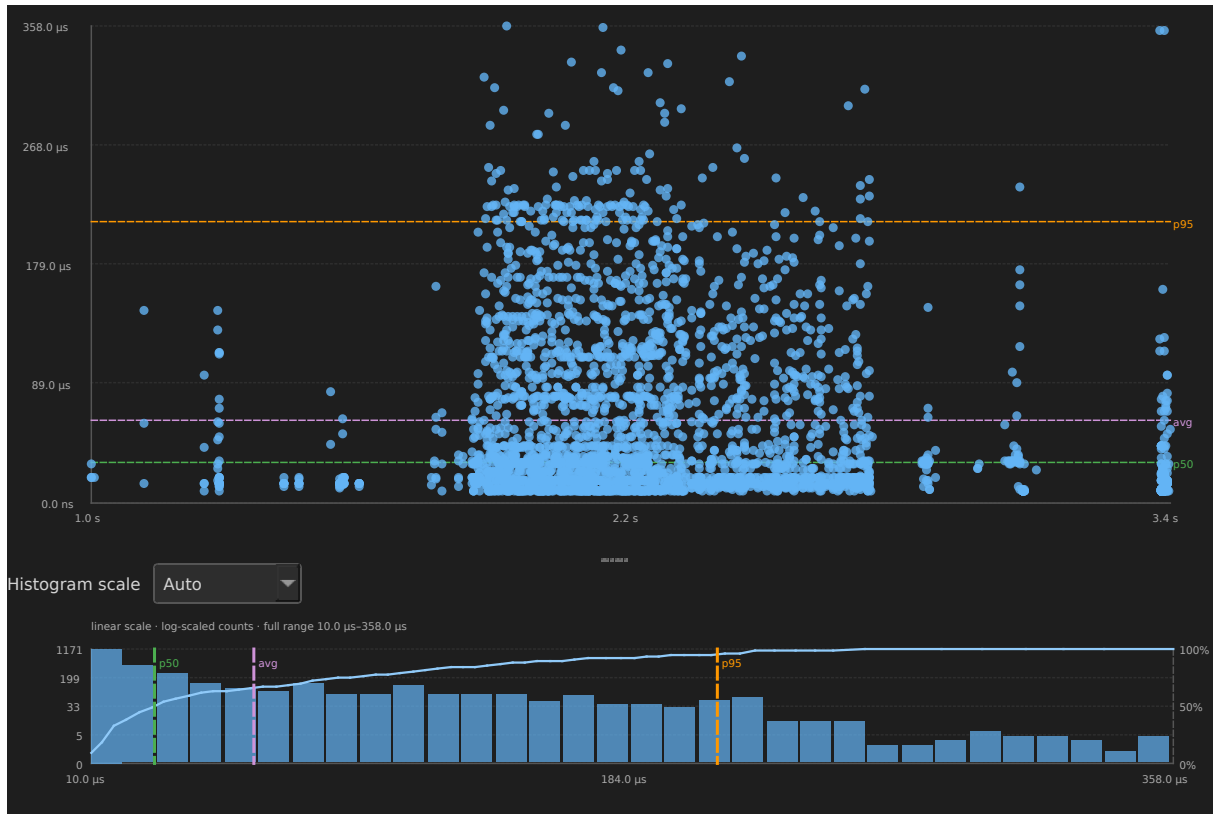


Figure 3: Concurrent core active interval-duration distribution for N=4 in example-8cores.btf.gz

Kernel Switch Overhead Per-core cost of consecutive context switches from core_segs gaps — time from the previous slice’s preempt/end to the next slice’s resume/start on the same core:

$$O_{\text{switch}} = t_{\text{resume},B} - t_{\text{preempt},A}$$

Column	Meaning
Core	Core name (Core_N)

Column	Meaning
Switches	Number of consecutive-slice gaps in scope
Min / Avg / Max	Switch-gap duration statistics
Total Overhead	Sum of gaps in scope
% of Core	Total overhead as a percentage of the core / scope span

Distribution chart — click any row: scatter of switch time vs gap duration; histogram with variability overlay ($\text{avg} \pm \sigma$). Zero or near-zero gaps mean back-to-back slices with negligible scheduler overhead in the trace resolution.

Headless snapshot (example-8cores.btf.gz, Core_0):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
  -o stats-switch-core0.svg --view plot --metric switch_overhead --core
  ↪ Core_0
```

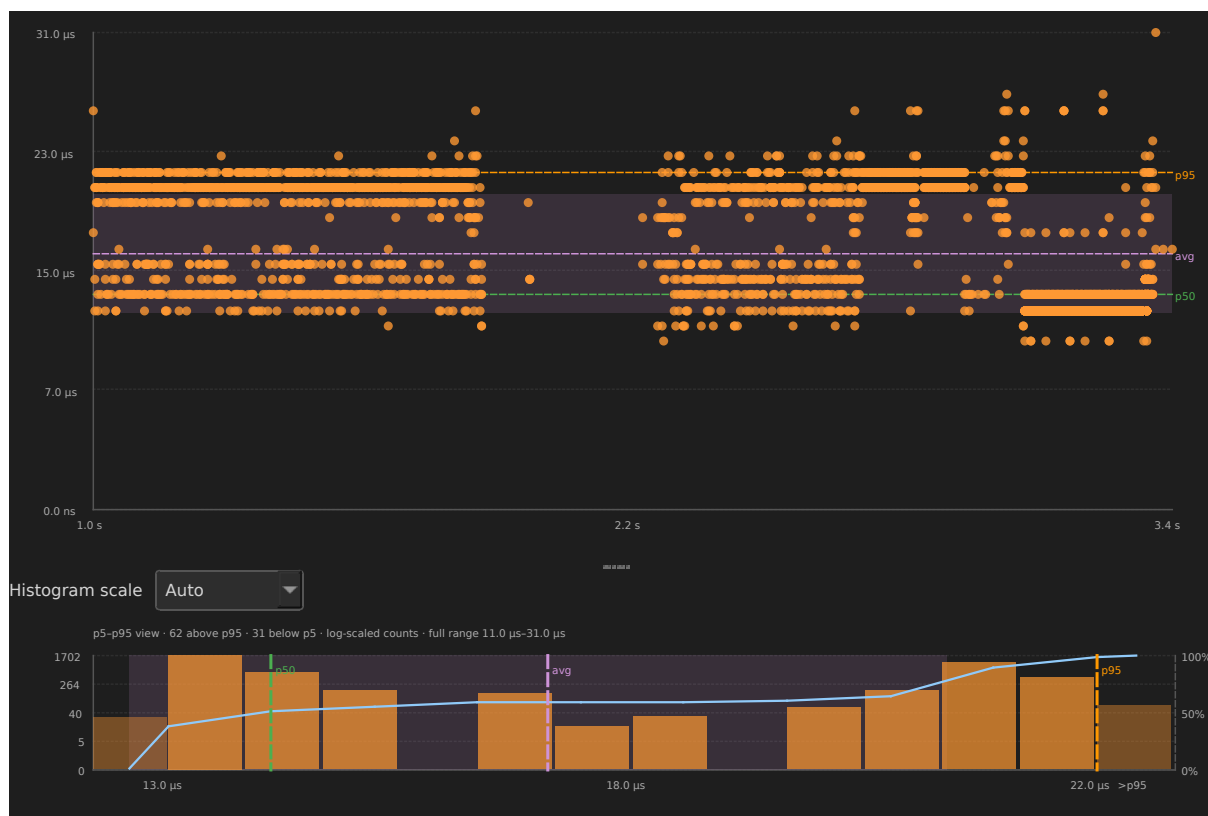


Figure 4: Kernel switch overhead distribution for Core_0 in example-8cores.btf.gz

Top tasks by CPU Ranks worker tasks (excl. IDLE/TICK) by total on-CPU time in scope — the same $T_{exec,i}$ denominator used for **CPU%** in Execution Time Per Slice. Click a row to highlight the task on the timeline.

Core Migrations → Deadlines (panel order) In the default panel sequence, the next sections are **Core Migrations**, **Core-Pair Migration Summary**, **Core Affinity**, **Task Lifecycle**, and **Deadlines / CPU budget**. Migration tables, heatmap, chord diagram, and **Trace Compare...** are documented together under Core migration analysis. Affinity, lifecycle, and deadline meanings are in the panel overview above and the subsections below.

Execution, blocking, and inter-arrival

These three task metrics are measured from consecutive on-CPU slices of the same task. The figure below shows how they relate on the timeline (UI label **Block Time** = Statistics **Blocking Time**):

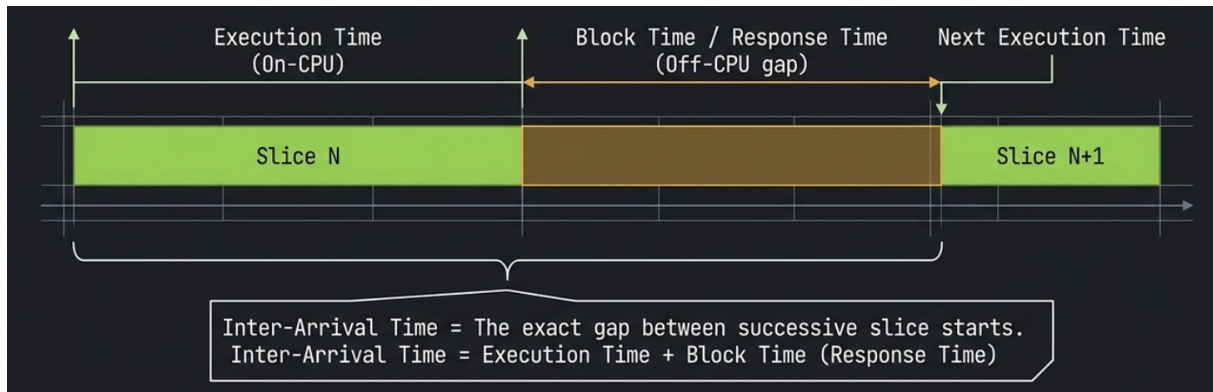


Figure 5: Execution Time, Block Time, and Inter-Arrival Time on consecutive task slices

Metric	Spans
Execution Time	Start → end of one on-CPU slice
Blocking Time	End of one slice → start of the next (off-CPU gap)
Inter-Arrival Time	Start of one slice → start of the next (period between activations)

Inter-arrival $\approx$ execution + blocking for the same pair of activations (when the gap is positive).

Execution Time Per Slice Measures how long each **on-CPU slice** lasts for a task — from switch-in until the task blocks, yields, or is preempted.

Formula — for task i , each on-CPU slice k in scope:

$$d_k = t_{\text{end},k} - t_{\text{start},k}$$

Table statistics are computed over all slice durations dk in scope. **Jitter** is the observed range, $\text{Max} - \text{Min}$; σ is the population standard deviation (the observed slices are treated as the complete population in scope). **CPU%** is the task's share of total active CPU time in scope:

$$\text{CPU}_i = \frac{T_{\text{exec},i}}{\sum_j T_{\text{exec},j}} \times 100$$

What it tells you: Short, uniform slices suggest periodic or tick-driven scheduling. A long **Max** or heavy **p95** tail marks worst-case execution time (WCET) slices — often critical sections, lock holds, or interrupt-disabled regions. Compare **Min** (BCET) and **Max** (WCET) to judge jitter; a wide spread on a real-time task may violate deadline assumptions even when **Avg** looks acceptable.

Column	Meaning
Task	Display name (Name[id])
Runs	Number of slices in scope
CPU%	Share of total trace (or cursor-range) active time
Min / Avg / Max / p95	Slice duration statistics
Jitter	Observed duration range ($\text{Max} - \text{Min}$)
σ	Population standard deviation of slice durations
Min / Max links	Jump and annotate BCET / WCET slice

Distribution chart — click any row:

- **Scatter:** x = slice start time, y = slice duration.
- **Histogram:** distribution of slice durations (auto log scale when the tail is wide; the blue CDF curve shows the cumulative fraction of samples).
- **Variability overlay:** the translucent purple band is $\text{average} \pm \text{one population } \sigma$. Jitter ($\text{Max} - \text{Min}$) is the full extent of the plotted data, so it needs no marker lines.

In example-8cores.btf.gz, task **CS[11]** has 730 slices with a long tail of longer runs (context-switch stress tasks):

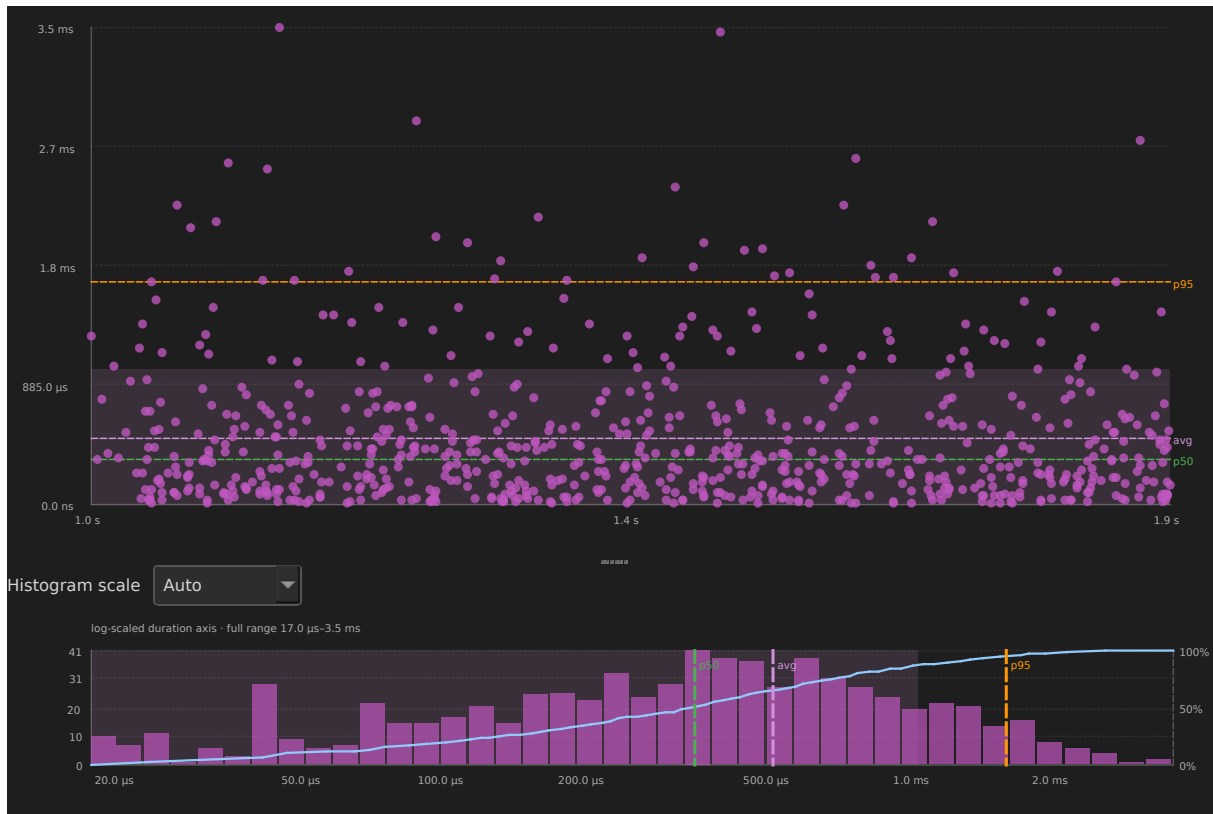


Figure 6: Execution time distribution for CS[11] in example-8cores.btf.gz

The scatter shows periodic bursts of short slices; the histogram uses a **log-scaled duration axis** so short and long slices are both visible. The **CDF** rises steeply on the left (most slices are short) then levels toward 100% as longer runs are included; **p50** and **p95** vertical markers align with the 50% and 95% ticks on the right axis.

Blocking Time Measures the **off-CPU gap** between the end of one slice and the start of the next for the same task — time spent waiting to run again (preempted, blocked on a resource, or delayed by the scheduler).

Not end-to-end response time: BTFViewer's **Blocking Time** is only the off-CPU gap until the task next resumes. True response time is normally measured from an explicit release event to completion and cannot be derived reliably from context-switch slices alone. It requires matching release/completion instrumentation (for example, paired interval events).

Formula — for consecutive activations k and $k+1$ of task i :

$$g_k = t_{\text{start},k+1} - t_{\text{end},k}$$

Only positive gaps are counted. Min / Avg / Max / p95 are taken over all gaps g_k in scope.

What it tells you: Blocking time is pure **wait time** — the task is runnable or blocked but not on-CPU. High **Avg** or **Max** gaps often point to lock contention, priority inversion, or a higher-priority task monopolizing the core. Spikes clustered at specific times in the scatter plot usually correlate with a particular preemptor or synchronization object; use **Preemption Chain Analysis** or **Mutex / Semaphore** pairing to find the cause.

Column	Meaning
Task	Display name
Gaps	Number of positive off-CPU gaps
Min / Avg / Max / p95	Gap duration statistics
Jitter	Observed gap range (Max – Min)
σ	Population standard deviation of off-CPU gaps
Min / Max links	Jump and annotate resume slice at shortest / longest gap

Distribution chart — click any row:

- **Scatter:** x = resume time, y = off-CPU gap.
- **Histogram:** distribution of blocking gaps.

CS[11] in `example-8cores.btf.gz` (729 gaps):

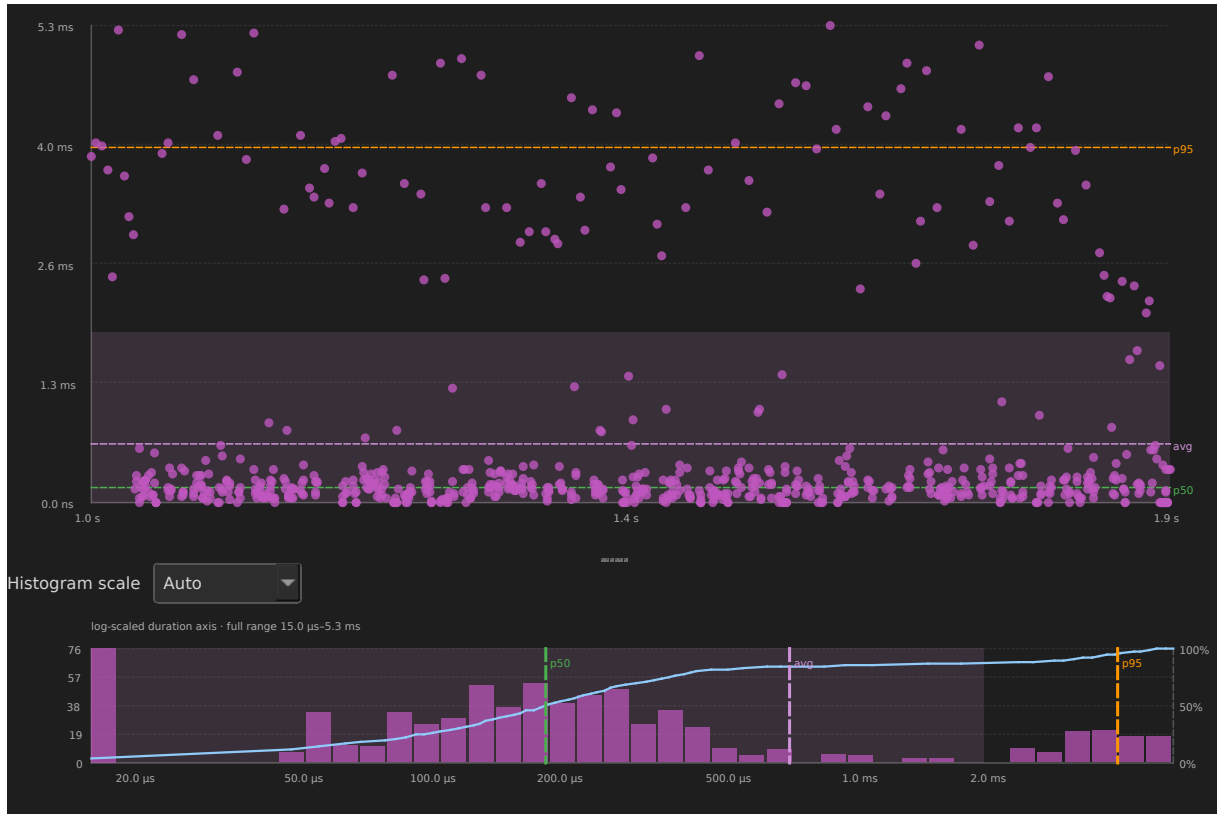


Figure 7: Blocking time distribution for CS[11] in example-8cores.btf.gz

High blocking gaps clustered at certain times often correlate with lock contention or a higher-priority task dominating the core.

Dispatch / Scheduling Latency Ready→run delay for a task: tready from STI resume Name[id] (vTaskResume) or task **create** time, and tresume from the next switch-in (segment start):

$$L_{\text{dispatch},k} = t_{\text{resume},k} - t_{\text{ready},k}$$

Sync-object wakes (give/send) are **not** attributed yet — BTF notes carry the object pointer, not the woken task id.

Column	Meaning
Task	Display name (Name[id])
Activations	Number of dispatch samples in scope
Min / Avg / Max / p95	Dispatch latency duration statistics
Jitter / σ	Observed range (Max – Min) and population standard deviation
Min / Max links	Jump and annotate the fastest / slowest dispatch

Distribution chart — click any row: scatter of dispatch time vs latency; histogram with variability overlay. Click a point to jump to the switch-in segment.

Headless snapshot (example-8cores.btf.gz, lifecycle test task **SR0[271]** — create / suspend / resume samples):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
  -o stats-dispatch-sr0.svg --view plot --metric dispatch --task
  ↪ "SR0[271]"
```

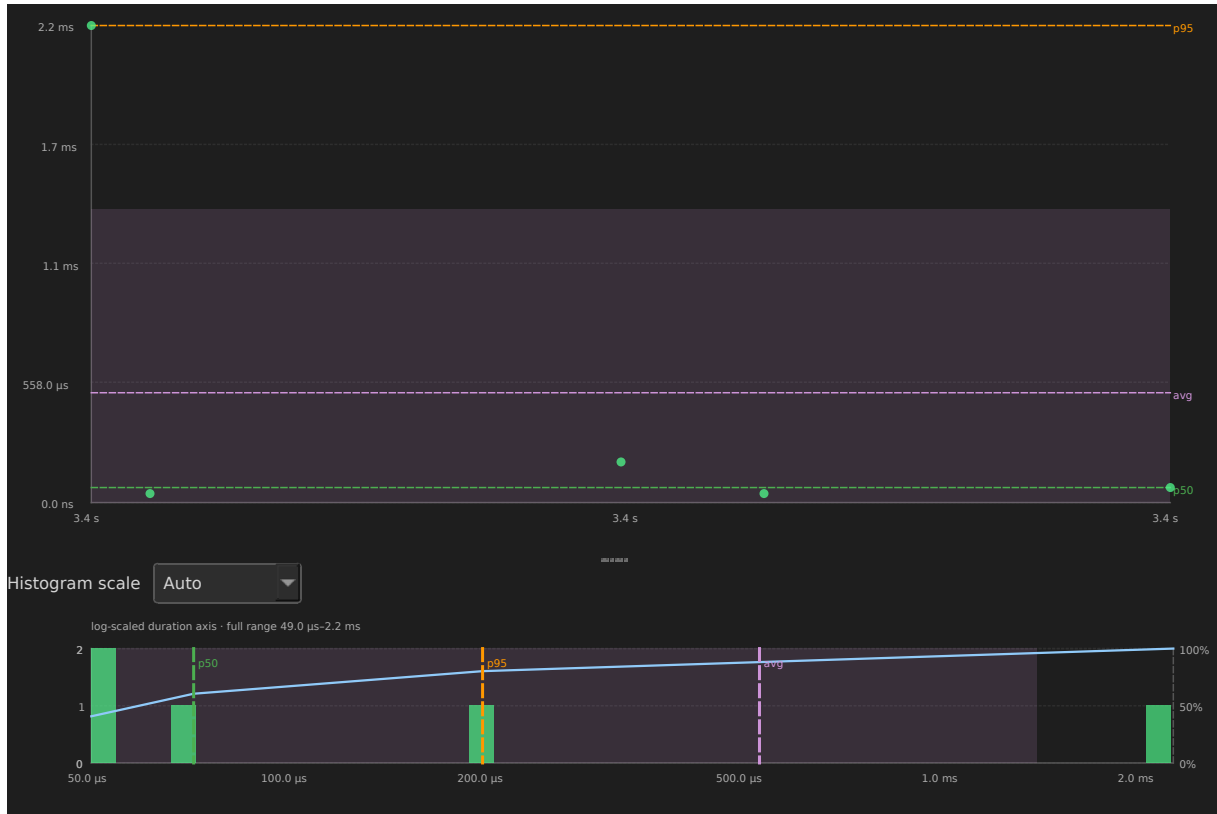


Figure 8: Dispatch latency distribution for SR0[271] in example-8cores.btf.gz

Inter-Arrival Time Measures the gap between **successive activation start times** of the same task (time between slice starts, not off-CPU gap).

Formula — for consecutive activations k and $k+1$:

$$\Delta t_k = t_{\text{start},k+1} - t_{\text{start},k}$$

Min / Avg / Max / p95 are taken over all inter-arrival samples Δt_k in scope.

What it tells you: Inter-arrival time reflects how often the task is **scheduled to run**, including time it spent on-CPU. For periodic tasks it should cluster near the expected period; drift or bimodality hints at missed deadlines, timer jitter, or workload-dependent release patterns. Because $\Delta t_k = dk + gk$ (slice duration plus blocking gap), inter-arrival is always $\geq$ blocking time for the same activations — compare both tables to see whether jitter comes from short runs or long waits.

Column	Meaning
Task	Display name
Runs	Number of inter-arrival samples
Min / Avg / Max / p95	Gap between activation starts
Jitter	Observed inter-arrival range (Max – Min)
σ	Population standard deviation of inter-arrival times
Min / Max links	Jump and annotate activation at shortest / longest inter-arrival

Distribution chart — click any row:

- **Scatter:** x = activation time, y = gap since previous activation.
- **Histogram:** distribution of inter-arrival gaps.

CS[11] in `example-8cores.btf.gz`:

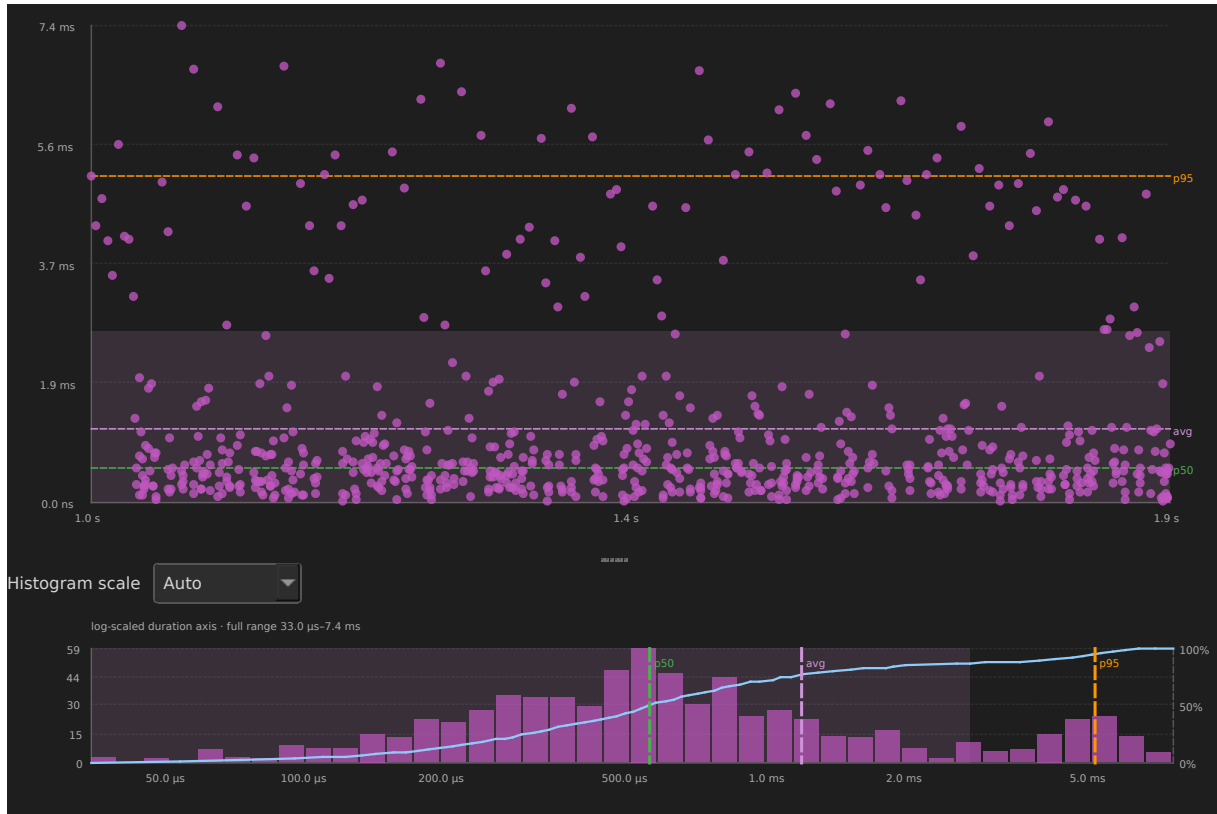


Figure 9: Inter-arrival time distribution for CS[11] in example-8cores.btf.gz

Compare with Blocking Time: inter-arrival includes time the task was **running**, so values are typically larger than off-CPU gaps alone. The histogram auto-selects **log duration** for CS[11] because activation gaps span microseconds to milliseconds.

Preemption Chain Analysis For each **victim** task's off-CPU gap, the analyser finds which **preemptor** tasks ran on the **same core** as the victim during that gap and aggregates overlap duration.

Formula — for victim v and preemptor p on the same core during gap g :

$$\text{overlap}(v, p, g) = \sum_{p \in g} [\min(t_{\text{end}}, g_{\text{end}}) - \max(t_{\text{start}}, g_{\text{start}})]$$

Count is the number of such overlap events; **Total** / **Avg** / **Max** summarise overlap durations for each victim←preemptor pair.

What it tells you: Answers *who ran while this task waited*. A single pair with high **Count** and **Total** means one preemptor dominates the victim's blocking; many pairs with moderate counts suggest fair sharing or frequent context switches. Large **Max** overlap points to long stretches where the victim was ready but could not run — a common sign of priority misconfiguration or a CPU hog.

Column	Meaning
Victim	Task that was off-CPU
Preemptor	Task that ran during the victim's gap
Count	Number of preemption overlap events
Total / Avg / Max	Overlap duration (how long the preemptor held the CPU during victim gaps)

Distribution chart — click any row (victim ← preemptor pair):

- **Scatter:** x = when the preemption overlap started, y = overlap duration.
- **Histogram:** how long that preemptor typically held the CPU during gaps.
- **Click a point** to jump to the **preemptor's segment** and add an annotation with duration/time notes.

CS[24] ← CS[25] in `example-8cores.btf.gz` (55 overlap events, 14.936 ms total overlap) — two context-switch stress tasks repeatedly preempting each other:

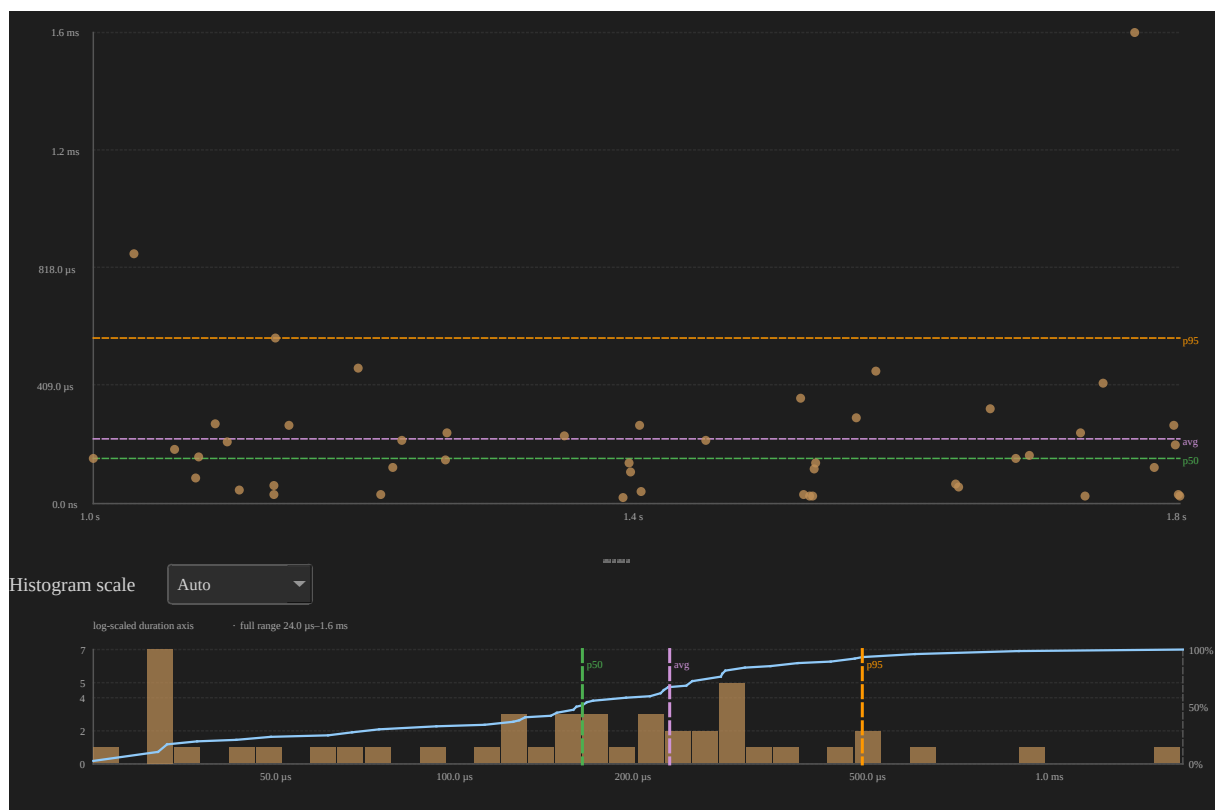


Figure 10: Preemption chain distribution CS[24] preempted by CS[25] in example-8cores.btf.gz

High **Count** with moderate **Avg** overlap suggests frequent short preemptions; a few points with large **y** values are long stretches where CS[25] ran while CS[24] waited. Use this table to answer *who preempted whom* and whether a victim's blocking is dominated by one preemptor or many.

Priority Inheritance Shown when the trace has **create pri:N** on task-create T rows **and** at least one priority STI on the task channel:

STI note prefix	Hook	Meaning
priority_inherit Name[id] pri:N	traceTASK_PRIORITY_INHERIT	Priority inheritance inherited priority N

STI note prefix	Hook	Meaning
priority_disinherit Name[id] pri:N	traceTASK_PRIORITY_DISINHERIT	Mutex_DISINHERIT returned to base priority <i>N</i>
set_priority Name[id] pri:N	traceTASK_PRIORITY_SET	EXPICIT vTaskPrioritySet()

Formula — a **boost episode** is a contiguous interval where the task's effective priority is above its **Base** (from create pri:N):

$$T_{\text{boosted}} = \sum_{\text{episodes}} (t_{\text{end}} - t_{\text{start}})$$

where *tstart* is the inherit / set-up STI and *tend* is the disinherit / set-back STI for each episode.

Boosts counts episodes; **Peak** is the highest priority observed while boosted.

What it tells you: Priority inheritance prevents priority inversion when a low-priority mutex holder blocks a high-priority waiter. Long **Boosted** time or many **Boosts** on a task that is not the mutex owner under test may indicate lock contention or chained inheritance. **Mutex inherit** confirms the kernel raised priority via `priority_inherit`; **L/M/H pattern** flags the classic three-level inversion geometry (see below); **Boost only** means manual `vTaskPrioritySet()` without mutex hooks.

Column	Meaning
Task	Mutex holder or subject whose priority was raised
Base	Priority at task create (create pri:N)
Peak	Highest set_priority level observed while boosted
Boosts	Number of boost episodes (base → above base → back to base)

Column	Meaning
Boosted	Total time above base priority in scope
Pattern	Summary label — see L/M/H pattern and table below

Pattern (summary)	When it appears
Mutex inherit	At least one boost episode used <code>priority_inherit / priority_disinherit</code> , and no extra inversion episodes beyond inherits
Mutex inherit + L/M/H	Mutex inheritance and additional boost episodes where medium-priority tasks sit between base and peak
L/M/H pattern	Boost above base (usually <code>set_priority</code>) with medium-priority tasks between Base and Peak , but no <code>priority_inherit</code> on that task
Boost only	Raised above base with no medium tasks in range and no mutex inherit

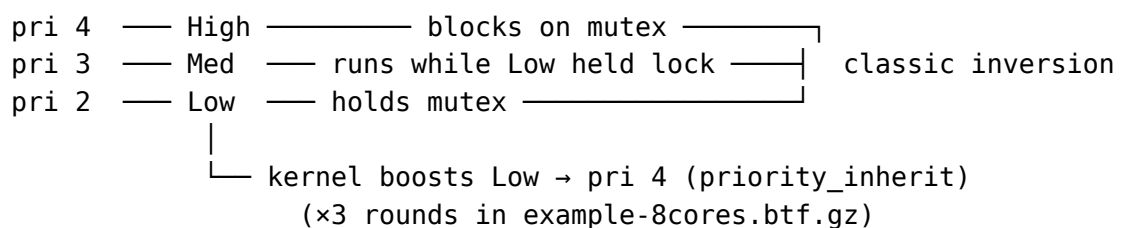
Per-episode detail (distribution chart tooltips, **Export HTML → Boost episodes**) can be more specific, e.g. `Mutex inherit L/M/H (CS[11], CS[12] +126)` — listing up to two medium-task names plus a count of any others.

L/M/H pattern (priority inversion geometry) **L/M/H** names the textbook **priority-inversion** layout on three priority levels. In the demo firmware (Demo/examples/freertos_test/main.c test 8) the tasks are named **Low**, **Med**, and **High**, pinned to **Core_0** on SMP, and the scenario is repeated **T8_ROUNDS (3)**

times so the timeline shows multiple red boost stripes and the Priority Inheritance chart has several inherit points:

Role	Meaning	In example-8cores.btfgz test 8
L (Low)	Mutex holder at the lowest priority of the three	Low[266] — create pri:2, holds the test mutex
M (Medium)	Runnable work at a priority between L and H; can preempt L while H waits	Med[267] — create pri:3, CPU work after Low takes the mutex
H (High)	Blocked on the mutex L holds; triggers inheritance	High[268] — create pri:4, blocks on the same mutex

Without mutex priority inheritance, **M** would run while **H** waited on **L** — unbounded inversion. FreeRTOS boosts **L** to **H**'s priority (e.g. first priority_inherit Low[266] pri:4 near ~3099 ms) so **L** finishes its critical section before **M** can starve **H**.



How the viewer detects L/M/H: at the end of each boost episode, it scans every task with a known create pri:N. Any task whose **base priority** satisfies **Base** < *p* < **Peak** (strictly between the boosted task's base and peak) is a **medium blocker**. If at least one exists, the episode is an **inversion suspect** and contributes to the **L/M/H** pattern.

- **Episode** level: inversionSuspect when the episode was mutex-inherited **or** medium blockers exist; episode **Pattern** may list medium task names.

- **Summary** level (table **Pattern** column): aggregates episodes — Mutex inherit, L/M/H pattern, Mutex inherit + L/M/H, or Boost only.

Important: medium-blocker detection uses **all** tasks in the trace with recorded create priorities, not only tasks active in that instant. On a busy SMP trace (many workers at pri:3 from earlier tests), the episode label may show several medium names (e.g. Mutex inherit L/M/H (CS[11], CS[12] +126)). For test 8, **Med[267]** is the semantically relevant **M**; cross-check with the timeline around test 8 (~3042-3359 ms absolute, --lo 3042000 --hi 3359000) and Demo/examples/freertos_test/main.c test 8 (vInvLow / vInvMed / vInvHigh).

Example — test 8 in example-8cores.btf.gz (Low[266]):

Field	Value
Base / Peak	2 → 4
Boosts / Boosted	3 mutex-inherit episodes, ~34.4 ms each (~103 ms total)
Summary Pattern	Mutex inherit
Episode Pattern	Mutex inherit L/M/H (medium tasks include Med[267] at pri 3)
STI windows	3099448-3133873, 3187403-3221850, 3275380-3309826 µs

Zoom the timeline to that window (or click a **Low[266]** scatter point) to see the **three red** boost stripes on the Low row and **High** blocking on the mutex between them.

Contrast — test 7 (PS[228], manual boost): the runner calls vTaskPrioritySet(subject, BOOST_PRIORITY) — trace shows set_priority STI events, not priority_inherit. **PS[228]** has the same numeric geometry (base 2, peak 4, medium fillers at pri 3) so the summary **Pattern** is **L/M/H pattern** (not **Mutex inherit**). Boost duration is much shorter (~119 µs) because no mutex hold loop is involved. Use **PS[228]** vs **Low[266]** to separate kernel inheritance from application-driven priority changes.

Timeline UX — boosted periods appear as a **bottom stripe** on the task row (horizontal) or a **right-edge stripe** (vertical): **orange** = boost only (Boost only /

manual set_priority without L/M/H geometry); **red** = mutex inherit or any L/M/H-related pattern. Task labels show **• pri N** when create priority is known.

Low[266] on the timeline in example-8cores.btf.gz (zoomed to test 8, --lo 3042000 --hi 3359000). Three **red bottom stripes** on the Low row mark the priority_inherit → priority_disinherit windows (~34.4 ms each). **Med[267]** runs on the same core before each inherit; during each boost window Med stays off-CPU:

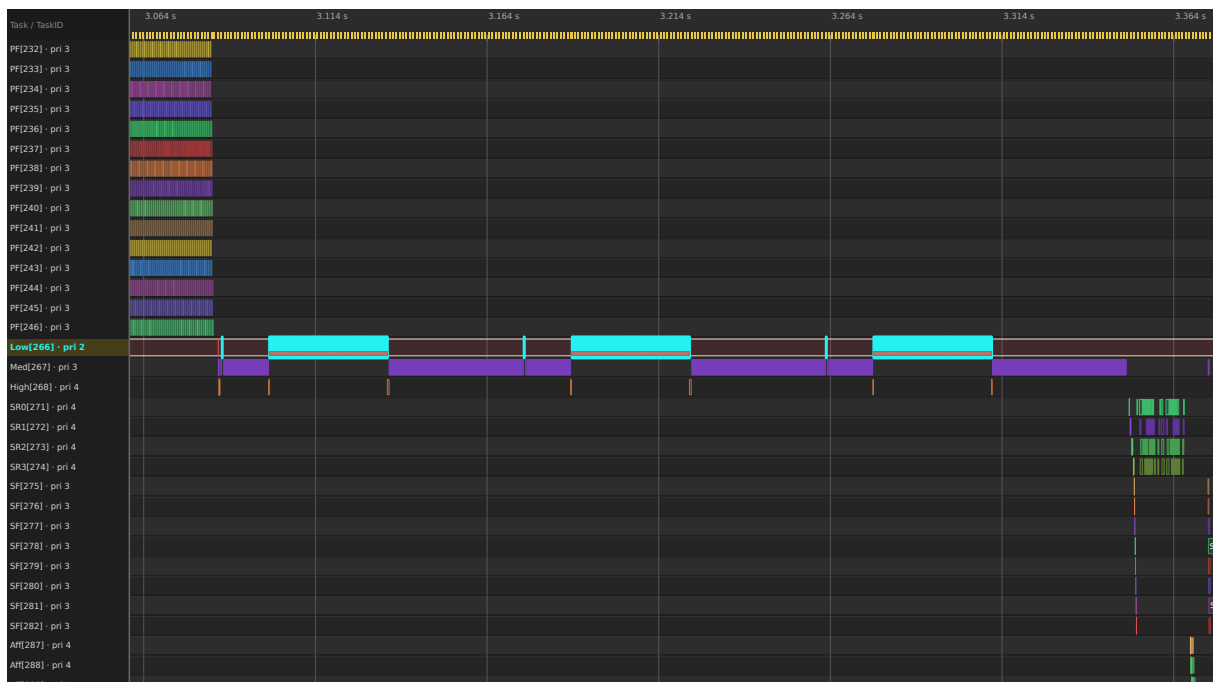


Figure 11: Timeline view: Low[266] with three red priority-inheritance stripes (example-8cores.btf.gz)

Export a timeline SVG from the viewer (**File → Save SVG** or toolbar) after zooming to the episode, or regenerate it headlessly via `make -C BTFViewer update-images` (desktop CLI snapshot `--view timeline --task ... --lo ... --hi ...`).

Distribution chart — click any Priority Inheritance table row:

- **Scatter:** x = boost episode end time, y = boosted duration. Orange points = boost only; red = L/M/H / mutex inherit. Test 8 contributes **three** clustered red points near ~34 ms.
- **Histogram:** distribution of boost durations.

- **Click a point** to zoom to that episode, scroll to the task row, highlight it, and add an annotation (re-click skips duplicate annotations).

Export HTML includes a **Boost episodes** detail sub-table (up to 200 rows by start time).

Low[266] distribution (test 8: three mutex inherit episodes, base pri 2 → peak 4, ~34.4 ms each):

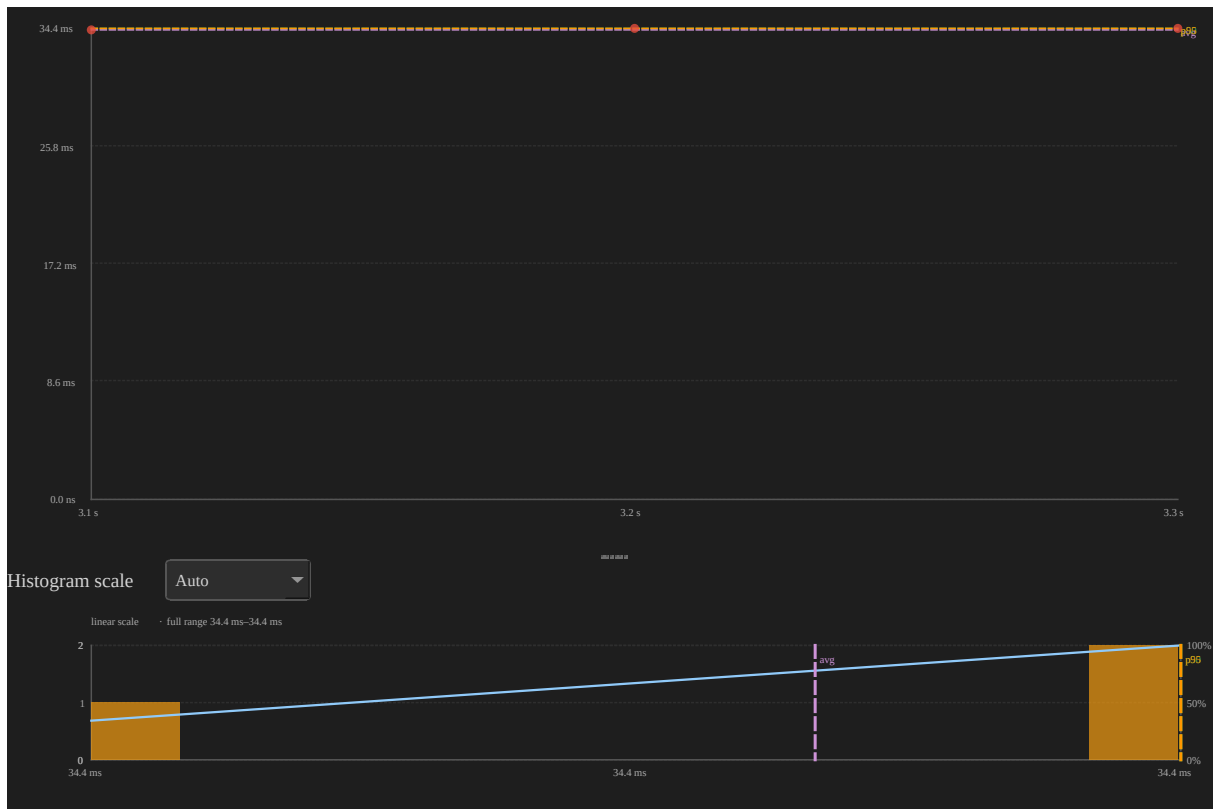


Figure 12: Priority boost distribution chart for Low[266] in example-8cores.btf.gz

PS[228] (test 7: manual `vTaskPrioritySet`, **L/M/H pattern**, ~119 μ s boosted) contrasts with **Low[266]** above — same base/peak numbers, different mechanism and duration.

Note: `traceTASK_PRIORITY_INHERIT` / `traceTASK_PRIORITY_DISINHERIT` are invoked by the FreeRTOS kernel inside `xTaskPriorityInherit()` / `xTaskPriorityDisinherit()` when `configUSE_MUTEXES` is enabled.

Mutex / Semaphore pairing When queue trace hooks are enabled (`configINCLUDE_QUEUE_EV`), mutex and semaphore STI lines include the **FreeRTOS object pointer** in the note:

```
703266,Core_0,0,STI,mutex,0,trigger,take 0x80018700
707222,Core_3,0,STI,mutex,0,trigger,give 0x80018700
```

The viewer pairs **take/give** (and **create/delete**) **per pointer** — not per STI channel alone — so two mutexes of the same type stay distinct. The kernel **give** immediately after **create** (mutex / binary sem available) is ignored when it falls within **1 ms** of the create event.

Semaphores use two pairing directions automatically:

Pattern	Example	Pairing
Hold (take → give)	Area slot sem, worker acquires then releases	take opens, matching give closes (FIFO)
Signal (give → take)	*_done / *_go coordination sems	give posts, matching take consumes (FIFO)

Mutexes always use hold pairing (LIFO — owner must give).

Formula — for each paired hold span h of duration τ_h :

$$\bar{\tau}_{\text{hold}} = \frac{1}{N_{\text{holds}}} \sum_h \tau_h$$

What it tells you: **Avg hold** shows typical lock or semaphore residency time — very long holds inflate blocking for waiters. **Issues** > 0 (orphan give, cross-task give, unmatched take, delete while held) mean the trace does not form clean take/give pairs and hold statistics may be incomplete. **Deadlock risk** at trace end flags multiple mutexes still held by different tasks — verify whether that is expected teardown or a real stall.

Column	Meaning
Object	Kind + pointer (mutex 0x80018700)

Column	Meaning
Kind	mutex or sem
Holds	Number of paired take→give spans in scope
Issues	Pairing problems in scope
Avg hold	Mean hold duration across paired spans
Status	OK, Warning, or Error

Check	Severity	Meaning
Orphan give	Error	Mutex give with no matching open take on that pointer
Cross-task give	Warning	Mutex give by a different task than the one that take
Unmatched take	Warning	take still open at trace end
Unmatched give	Warning	Semaphore give still unmatched at trace end
Delete while held	Warning	delete while resource takes are still open (common during teardown)

Check	Severity	Meaning
CORE_MIGRATION_WHILE_HELD	Warning	Mutex take on one core, give on a different core — the lock crossed a core boundary while held; indicates a cache-line bounce where the hardware cache line containing the lock data is transferred between cores. Shown in the Pairing Issues sub-table with the exact from/to cores (e.g. Lock bounced from Core_0 to Core_1)
Deadlock risk	Warning	≥ 2 mutexes still held by ≥ 2 different tasks at trace end

The running task for each event is inferred from the **core timeline** at that timestamp (same approach as interval tid pairing).

Below the summary table, a **Pairing issues** sub-table lists every problem in scope (time, object, issue kind, detail). **Click any issue row** to zoom to the running task segment on that core (when found), jump to the issue timestamp, highlight the segment, and add an annotation with a descriptive note (Desktop + Web). Re-clicking the same point skips duplicate annotations.

Export HTML adds two detail sub-tables under this section: all **Pairing issues** in scope (including CORE_MIGRATION_WHILE_HELD warnings), and **Hold episodes** (longest first, up to 150 rows) with **Take core** and **Give core** columns.

Export CSV includes a **Core Affinity Violations** sub-section listing each mutex that had at least one bounced hold, with the bounce count and a description.

example-4cores.btf.gz (tests 1-3) exercises 0x80018700 (mutex) and 0x80018650 (counting sem) with clean hold pairing. The full trace shows CORE_MIGRATION_WHILE_HELD

warnings on 0x80018700 (3 bounces) and 0x80018650 (1 bounce) — the Statistics **Mutex / Semaphore** summary table will show **Warning** for these mutexes and non-zero values in the **Bounces** column. Coordination sems pair in **signal** direction.

Queue When queue STI events are present (configINCLUDE_QUEUE_EVENTS), the **Queue** section pairs send/recv (and create/delete) **per object pointer**, the same way Mutex / Semaphore pairing works for take/give.

Column	Meaning
Object	Kind + pointer (queue 0x.....)
Holds	Number of paired send→recv (or equivalent) spans in scope
Issues	Pairing problems in scope
Avg hold	Mean hold duration across paired spans
Status	OK, Warning, or Error

Shown only when the trace contains queue STI events. For mutex/semaphore bounce and issue detail, see **Mutex / Semaphore pairing** above.

Interval Analysis Pairs **interval_start** / **interval_stop** STI events into measurable code regions. Each interval **id** gets an **Interval N** row on the timeline (horizontal task view) with colored span bars; the statistics table aggregates duration across all paired spans for that id.

Formula — for each paired instance j with start t_s and stop t_e :

$$\tau_j = t_e - t_s$$

Count is the number of paired spans in scope; Min / Avg / Max / p95 are over all interval durations τ_j .

What it tells you: Interval metrics measure **how long instrumented code regions take** — loop iterations, critical sections, or end-to-end handlers. Tight clusters in the distribution chart mean stable iteration time; outliers or a high **Max** often mark contention, preemption inside the region, or pairing artefacts (see **Limitations** under Interval Analysis below). Compare interval ids to separate workloads (e.g. mutex stress vs lighter loops in the same trace).

Cross-task timing tip: `interval_start / interval_stop` pairing is scoped **per task id** (see Pairing algorithm below) — it measures elapsed time **within the same task**. To measure elapsed time **between two different tasks or ISRs** (e.g. a producer marks a tag on one task/core and a consumer picks it up on another), pairing by task id doesn't apply. Use a **tag channel** (`btf_traceTAG(id, value)`) instead: tag samples carry no task/id pairing, so consecutive samples on the same channel — regardless of which task emitted them — measure the cross-task interval directly. See Tag Analysis → Interval tab.

BTF note field (last CSV column on each interval line):

Format	Example	Viewer pairing
Current firmware	1 tid:7 or 0 tid:0x8	Interval id + task id (decimal or 0x hex); same numeric id on different tasks does not cross-pair
Legacy	1	Full note string 1 only

Recorded by `traceINTERVAL_START(id) / traceINTERVAL_STOP(id)` in firmware — task id is captured automatically in param2 and emitted in the note as `tid:...`. See Binary → BTF dump mapping in the repo root README.

Column	Meaning
ID	Interval id from <code>traceINTERVAL_START(id) /</code> <code>traceINTERVAL_STOP(id)</code>
Label	Display name (Interval N)

Column	Meaning
Count	Number of paired start→stop spans in scope
Min / Avg / Max / p95	Interval duration statistics

Distribution chart — click any row:

- **Scatter:** x = interval stop time, y = interval duration.
- **Histogram:** distribution of interval durations.
- **Click a point** to jump to the **interval start** and add an annotation with duration/time notes.

Export HTML includes an **Interval instances** detail sub-table (longest first, up to 200 rows).

Interval 1 in `example-8cores.btf.gz` (1728 spans from eighteen vCtxSwitch-Worker tasks — CS[11]-CS[28] — sharing id 1 — each worker pairs via its own tid in the note):

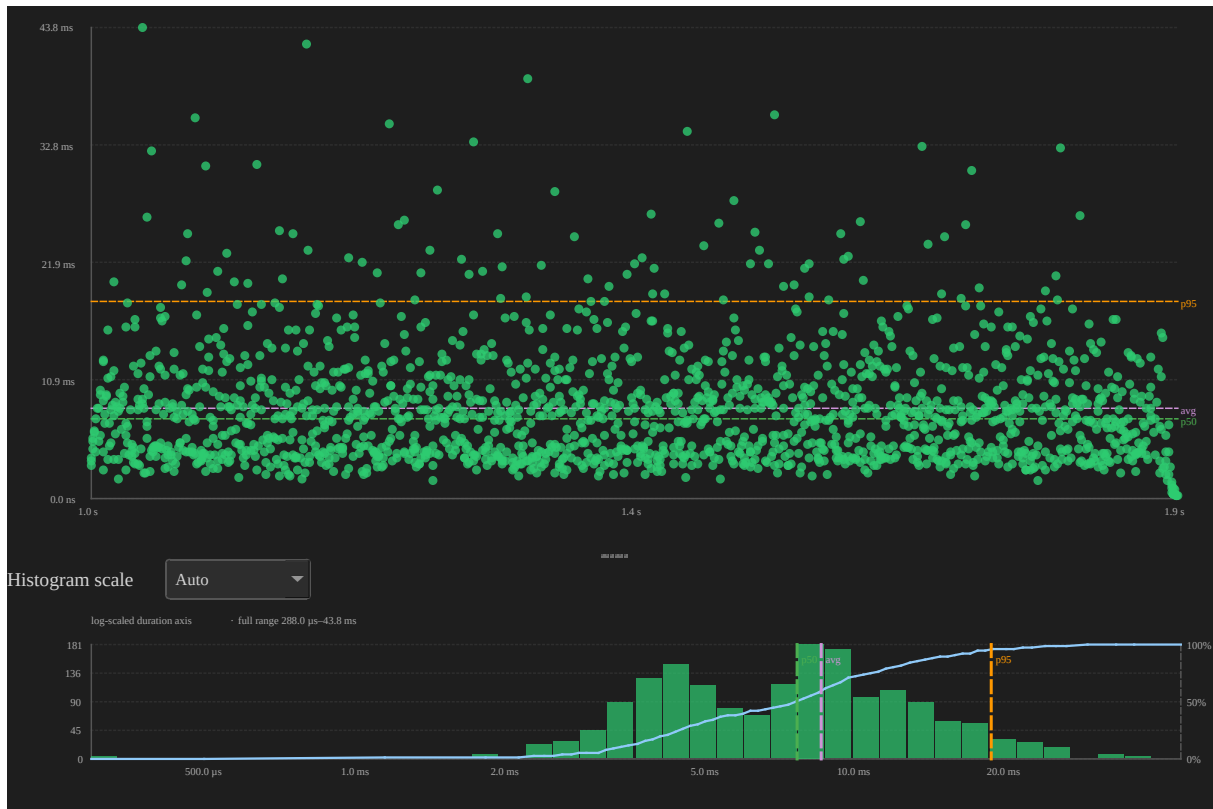


Figure 13: Interval duration distribution for interval id 1 in example-8cores.btf.gz

Most spans cluster at short durations (tight yield loops); occasional high **y** points mark iterations that waited longer on the shared mutex or area semaphore. Compare ids **4-6** (shorter per-iteration work) vs **1-3** (heavier stress tests) in the same table.

Pairing algorithm At parse time, the viewer collects all `interval_start` / `interval_stop` STI events, sorts them by time (start before stop at the same timestamp), and pairs them with a **per-key stack** (LIFO).

Note parsing (last BTF CSV field):

Note format	Pairing key	Timeline / stats row id
{id}	interval id +	id only (Interval N)
tid:{task_id} (current firmware)	task_id	
{id} or other legacy text	full note string	same as note

When `tid:` is present, concurrent workers can share the same interval **id** without cross-pairing: task 7's `START(1)` only pairs with task 7's `STOP(1)`, even if task 8 uses id 1 at the same time. Legacy traces without `tid` keep the original behaviour (pair by note / id string only).

1. **interval_start** — push the event onto that key's stack.
2. **interval_stop** — pop the most recent unmatched start for that key and form one instance `[start_time, stop_time]`.
3. **interval_stop with an empty stack** — ignored (orphan stop).
4. **Unmatched starts after the trace ends** — counted internally but not shown in statistics or on the timeline.

An **Interval N** row appears only when at least one **start→stop** pair exists for that id. If firmware emits `interval_start` without matching `interval_stop` events (same id and `tid` when present), that id is omitted entirely.

Example — interval id with no paired spans:

Interval id	interval_start	interval_stop	Viewer
0	50	50	Interval 0
1	100	0	<i>(no row — nothing to pair)</i>
2	50	50	Interval 2

If firmware emits `interval_start` for an id but never records matching `interval_stop` events (same id and `tid` when present), that id is omitted from the timeline and statistics. Ensure every `traceINTERVAL_START(id)` has a corresponding `traceINTERVAL_STOP(id)` with the same task id in the note when `tid:` is used.

This is **LIFO (last-in, first-out) nesting** within each pairing key.

```
# Nested on one task (same pairing key)
START(id=2)          stack: [A]
  START(id=2)        stack: [A, B]
  STOP(id=2)   →     pairs B→B (inner span)
STOP(id=2)   →     pairs A→A (outer span)

# Two tasks, same interval id, with tid in note
Task 7: START  note=1 tid:7
Task 8: START  note=1 tid:8
Task 7: STOP   note=1 tid:7 → pairs with task 7 start only
Task 8: STOP   note=1 tid:8 → pairs with task 8 start only
```

Result: **two** instances — one for the inner region, one for the outer region — each with the correct duration. **Min / Avg / Max / p95** in the statistics table are computed over **all** paired instances whose time range overlaps the active scope (full trace or cursor range). Timeline display uses a separate rule (see below).

Limitations 1. Concurrent overlap with the same id (cross-pairing) — legacy traces without tid

When the BTF note has **no** tid:{task_id} suffix, pairing uses the note string only. The stack algorithm then assumes stops arrive in the **reverse order** of starts. That holds for true call-stack nesting on one thread, but **not** when several tasks use the **same interval id** at the same time.

```
Task on Core_1: START(2) ————— STOP(2)
Task on Core_2:   START(2) ————— STOP(2)
Event order:      S1          S2          S1          S2
                  └─ stack pairs S2 with S1's STOP (wrong)
```

When starts and stops interleave across cores, the viewer can pair one task's **start** with another task's **stop**. That produces:

- **Count** — still the number of stop events that found a start on the stack (often equals the number of loop iterations).
- **Min / Avg / Max / p95** — can be **wrong**: bogus **very long** spans inflate max and avg; the distribution chart shows outlier points far above the real iteration time.

example-4cores.btf.gz is recorded **with** tid in each interval note, so ids **1-4** pair correctly across parallel workers. **Legacy** traces that omit tid may still cross-pair when several tasks share one id:

Interval id	Test (see Demo/examples/freertos_test/main.c)	Without tid — typical symptom
1	Context-switch / mutex stress (vCtxSwitchWorker)	Inflated Max / outlier scatter points
2	Mutex workers (vMutexWorker)	Same
3	Area-semaphore workers	Same
4	Nested interval stress	Same

For legacy traces, treat **short-duration clusters** in the scatter/histogram as the meaningful iteration times; treat isolated **very large** points and the table **Max** as pairing artefacts unless you have verified LIFO ordering for that id.

2. SMP task migration — why pairing is not done by core

The viewer pairs by **interval id** (and **task id** when tid is present), not by the **core** field on each STI event. A core-based matcher might seem attractive when several tasks share one id, but on **SMP** it is **not reliable**:

- A single FreeRTOS task can **migrate** between cores while an interval is open (START on Core_0, body runs, STOP on Core_2 after migration).
- Preemption and scheduling can also make the “running core” at START/STOP differ from the core where most of the work ran.
- Matching START and STOP by core would then **fail to pair** valid spans or would **split one logical interval** into orphan events.

Task A (may migrate):

```
START(2) on Core_0 ... runs on Core_0, then Core_1 ... STOP(2) on Core_1
Core-based match: no single core owns both ends → broken or missed pair
Id + tid stack: pairs correctly if no other task uses id 2 concurrently
```

So the viewer does **not** use core as a pairing key. With **tid in the note**, several tasks may share the same interval id safely; without tid, prefer **one interval id per logical scope** or accept LIFO-only pairing. Core columns on paired instances (start_core / stop_core in the parsed data) are informational only.

3. True time nesting vs concurrent overlap

Situation	Pairing correct?	Statistics meaningful?
Nested START/STOP on one thread (LIFO order)	Yes	Yes — each nesting level is a separate instance
Concurrent tasks, different ids	Yes	Yes — ids are independent
Concurrent tasks, same id, with tid in note	Yes	Yes — per-task pairing keys
Concurrent tasks, same id, no tid (legacy)	Often no	Count ok; min/avg/max may be skewed
Start without matching stop (crash / trace cut-off)	Partial	Unmatched starts excluded from stats

4. Timeline bars vs statistics count

The statistics table and distribution charts use **every** paired instance in scope. The timeline draws **top-level spans only**: if instance B's [start, stop] lies entirely inside instance A's range (same id), only A is drawn. This is a **display** rule only; it does not change **Count** or min/avg/max.

View	What you see
Statistics → Count	All paired spans (including nested and cross-paired)
Distribution chart	One point per paired span (same set as Count)
Timeline → Interval N row	Non-nested spans only (fully contained children hidden)

Typical timeline bar counts for the full `example-4cores.bt f.gz` trace:

Interval id	Statistics Count	Timeline bars (top-level)
1	480	5
2	480	7
3	240	1
4	480	1

Interval 2 on the timeline: **7** bars (one long outer span plus six shorter top-level spans), while the table still reports **Count = 480**. Use the distribution chart (click a scatter point) or a narrow cursor range to jump to one specific paired instance.

5. Instrumentation guidance

To get reliable per-task interval statistics:

- **Preferred (firmware in this repo):** use `traceINTERVAL_START(id) / traceINTERVAL_STOP(id)` — the logger records `tid:{task_id}` in the BTF note so parallel workers can share the same numeric id.
- **Legacy traces** without `tid`: use a **distinct interval id per task** (or per logical scope), not one shared id across parallel workers.
- For nested regions on a single thread, reuse the same id — LIFO pairing matches `START/STOP` nesting within each pairing key.
- Do **not** rely on **core** to disambiguate pairs on SMP: tasks can **migrate** between `START` and `STOP` (see limitation 2 above).
- Orphan stops (stop without start) are dropped; orphan starts at end of trace are not counted in the table.

Timeline rendering Interval bars are drawn as **solid** spans in the interval's colour. **Start** and **stop** events are marked with vertical tick lines (solid at start, dashed at stop) on the interval row.

Tag Analysis Aggregates numeric samples from the 8 general-purpose STI **tag** channels (`tag0_event ... tag7_event`, plus the unindexed `tag_event`) — free-form

values emitted by firmware via `btf_traceTAG(id, value)` for any application-defined metric that doesn't fit an existing STI channel (queue depth, ADC reading, free heap, sensor reading, etc.). One row appears per channel that has at least one sample.

Formula — over all sample values v_k on a channel in scope: **Count** is the number of samples; **Min** / **Avg** / **Max** are the usual statistics; **p95** is the 95th-percentile value.

What it tells you: Unlike the other metric tables, the tag y-axis is the **raw application value**, not a duration — so what it means depends entirely on what firmware puts in the channel. A widening spread or a rising trend in the scatter plot can flag a slow memory leak (free heap), growing backlog (queue depth), or drifting sensor reading; a tight cluster around **Avg** with occasional **p95/Max** outliers is normal sampling noise.

Column	Meaning
Channel	Display name (Tag 0 ... Tag 7, or Tag for the unindexed tag_event)
Count	Number of samples in scope
Min / Avg / Max / p95	Sample value statistics (not a time duration)

Distribution chart — click any row:

- **Scatter:** x = sample time, y = tag value.
- **Histogram:** distribution of tag values.

Interval tab (time between samples) The distribution popup has two in-dialog tabs — **Value** (above) and **Interval** — mirroring the **Core Migrations** Dwell/Rate/Gap tabs. Click **Interval** to switch the same scatter + histogram widgets to show the **elapsed time between consecutive samples** on that channel, regardless of which task or core emitted them.

Formula — for consecutive samples (sorted by time) k and $k-1$ on one channel:

$$\delta_k = t_k - t_{k-1}$$

Scatter: x = the later sample's time, y = δ_k (rendered as a duration, using the same adaptive time units as other metric charts). **Histogram:** distribution of δ_k with the usual min/avg/max/p95 reference lines and CDF overlay. Clicking a scatter point jumps to and annotates the later sample's time.

This is the **recommended way to measure elapsed time across two different tasks or ISRs**: emit `btf_traceTAG(id, marker)` from each side of the cross-task event (e.g. once when a producer task hands off, once when the consumer task/ISR observes it) and read the interval distribution's min/avg/max/p95 as the cross-task latency — no tid pairing is needed since tag samples are unpaired, timestamp-ordered markers.

tag0_event in `example-8cores.btf.gz` (2330 samples, min 8,144, avg $\approx 36,845$, max 71,904, p95 41,936):

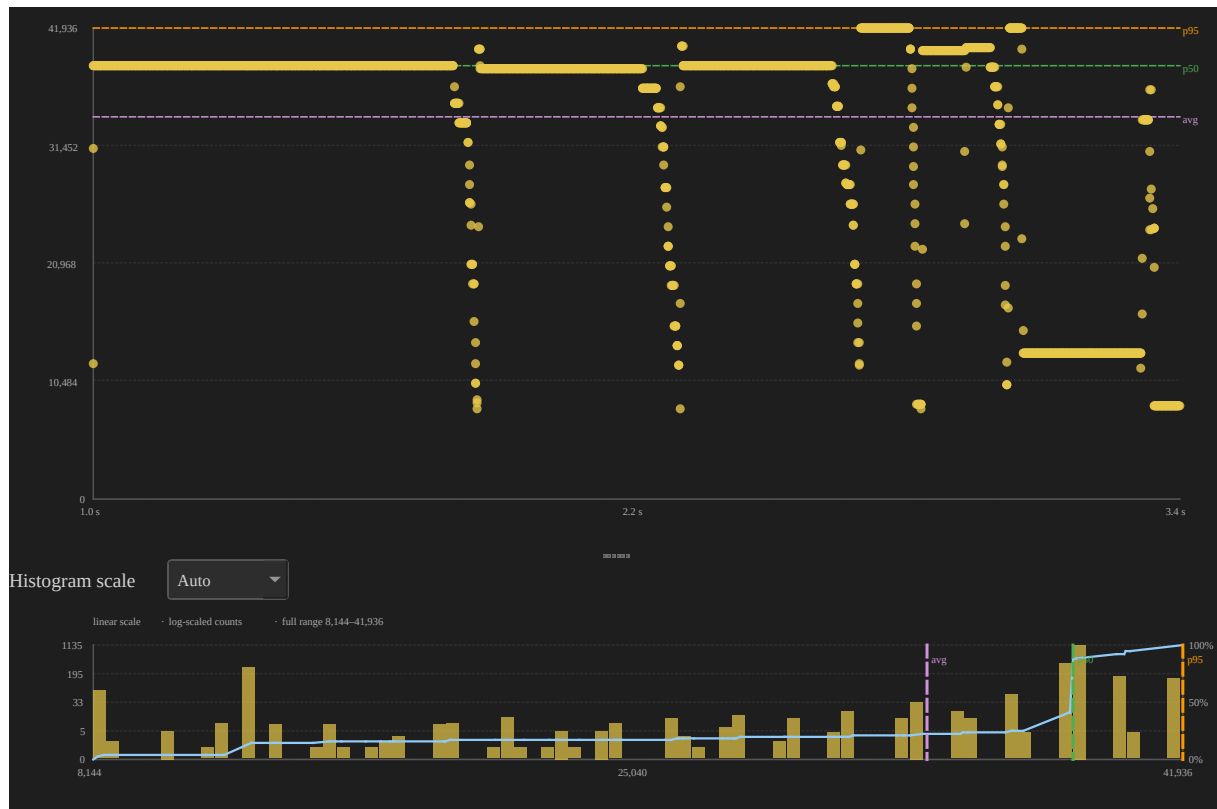


Figure 14: Tag value distribution chart for tag0_event in example-8cores.btf.gz

Shown only when the trace contains tag0_event ... tag7_event (or tag_event) STI samples.

Core migration analysis

A **migration** is recorded when consecutive slices of the same task (merge-key) run on different cores. Migrations are detected at parse time from the segment timeline — there are no separate markers drawn on the timeline; use the **Core Migrations** table, **Migration heatmap**, **Migration chord diagram**, **Trace Compare...**, or Find **Migrations** mode to inspect them.

In the Statistics panel default order, **Core Migrations** and **Core-Pair Migration Summary** sit after **Top tasks by CPU** (before affinity / lifecycle / deadlines). This section is the deep dive for those tables plus the heatmap and chord tools.

Highlight a migrating task on the timeline To see a hot migrating task in context (not only read rates in a table):

1. Stay in **Task View** (toolbar **Task**).
2. Click the task label (e.g. CS[22]) to **lock-highlight** it — other tasks stay on the timeline but gray out; do **not** use the legend filter.
3. Enable **Load** to see that task's CPU usage **per core** under the timeline (see CPU Load).
4. Optionally place cursors around the burst and enable **Limit to cursor range** so **Statistics → Core Migrations** and the heatmap recompute for that window only.

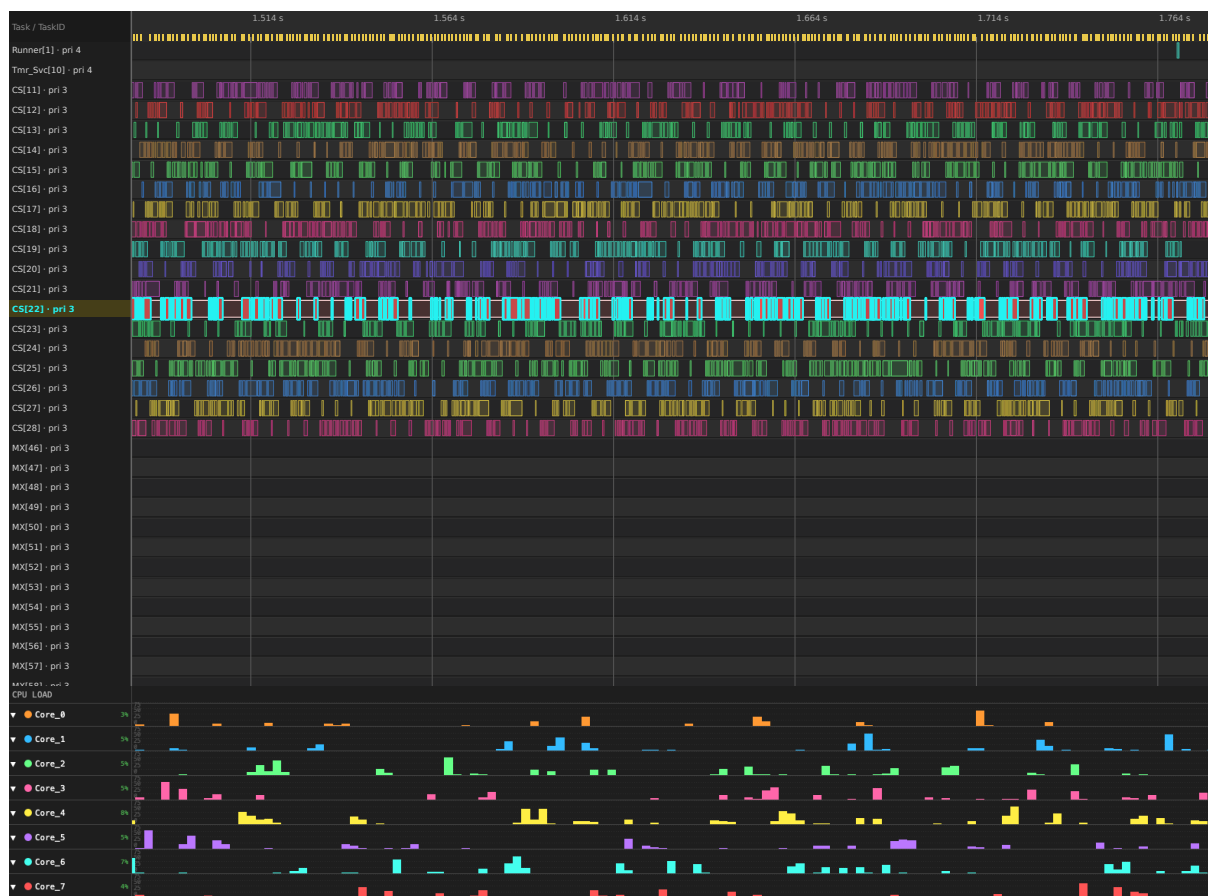


Figure 15: CS[22] highlighted in Task View with per-core CPU Load

CS[22] lock-highlighted in Task View; CPU Load shows that task's share on each core (example-8cores.btf.gz).

Headless example (full-span Task View + CPU Load for the same highlight):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o /tmp/cs22-cpu-load.svg \
--view timeline --view-mode task --task "CS[22]" --cpu-load
```

Zoomed burst without the load strip:

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o /tmp/cs22-burst.svg \
--view timeline --view-mode task --task "CS[22]" \
--lo 1805000 --hi 1865000
```

Inspect migrations involving a specific core After highlighting a hot task:

1. **Statistics → Core Migrations** — note **Primary** core, **Rate**, **Dwell**, **Ping** for that task. Click the row → **Dwell** / **Rate** / **Gap** charts.
2. **Core-Pair Migration Summary** — sort or scan pairs where that core is **From** or **To** (e.g. Core_5→Core_7) to see which neighbour absorbs the traffic and whether **Bounce %** is elevated. Click the row for Gap / Rate charts; use **Open Heatmap** / **Open Chord** from the dialog footer when you need time-bin density or topology.
3. Toolbar **Heatmap** — find the dark cells on rows that include the core of interest; click to drill to the per-task grid, then click the task cell to zoom/filter the timeline to that pair/window.
4. Toolbar **Chord** — hover the core's arc to highlight only its ingress/egress chords.
5. **Core Time Breakdown** — click a core row to jump Core View focused on that core.
6. Headless CSV for a scoped window:

```
python builds/btf_viewer.py migrations ../tracedata/example-8cores.btf.gz \
  --lo 1805000 --hi 1865000 -o - | head
```

Legend panel: check **Migrated tasks only** to hide tasks that never left their first core.

Statistics → Core Migrations (collapsible section) — tasks that ran on two or more cores:

Migration rate — normalizes raw migration count against task active time and (when TICK STIs exist) scheduler ticks, so a task that migrates often relative to how much it runs stands out:

$$R_m = \frac{N_{\text{migrations},i}}{T_{\text{exec},i}}$$

The **Rate** column shows R_m as migrations per second of on-CPU time (e.g. 1.23/s). When the trace includes TICK events, it also shows migrations per **on-CPU** scheduler tick for that task (e.g. 2.785/tick) — TICK STIs that fall inside one of the task's slices in scope, not trace-wide tick count.

What it tells you: A high rate means the task is **bouncing** between cores (thrashing). For high-priority real-time tasks you ideally want this close to zero.

Average core dwell time — mean duration of each on-CPU stay before the task blocks, yields, or migrates:

$$\bar{T}_d = \frac{1}{N_{\text{slices}}} \sum_k d_k = \frac{T_{\text{exec},i}}{N_{\text{slices},i}}$$

Each slice d_k is one switch-in episode (equivalent to averaging per-core dwell, $T_{\text{on}} / N_{\text{slices}}$, on each core the task visited).

What it tells you: If **Dwell** is extremely short (e.g. less than a few milliseconds or close to your system tick period), the scheduler is spending disproportionate effort moving the task between cores instead of letting it compute.

Column	Meaning
Task	Display name (Name[id])
Migr	Migration count in the current scope (full trace, or cursor range when Limit to cursor range is on)
Rate	Migration rate — /s of task active time; /tick = migrations per on-CPU TICK for this task in scope
Dwell	Average on-CPU slice duration (core dwell time) in scope
Cores	Distinct cores with on-CPU time or migrations in the current scope
Primary	Core with the most active time in scope, with its share (%)
Ping	Ping-pong migrations — three consecutive migrations A→B→A within 1 μs
STI±	Migrations with an STI event within ±500 ns

Column	Meaning
Gap after	Average off-CPU gap immediately after a migration
Gap other	Average blocking gap elsewhere for the same task

Click a row to open a **distribution chart** with three in-dialog tabs (switches the chart in place, no need to close and reopen):

- **Dwell** (default tab) — one point per on-core run: x = run start time, y = run duration dk (same definition as **Average core dwell time** above, but per-sample instead of averaged).
- **Rate** — one point per migration after the first: x = migration time, y = time since the task's *previous* migration. Clusters of short gaps mean bursts of rapid bouncing; a flat, evenly spaced scatter means migrations are rare and spread out.
- **Gap** — one point per migration with a positive **Gap after** value: x = migration time, y = off-CPU gap immediately following that migration (the same samples averaged into the **Gap after** column; **Gap other** is not plotted here — open the **Blocking Time** chart for the same task instead, since it covers all off-CPU gaps).

Each tab's scatter + histogram uses the same **adaptive scaling** as other metrics charts (see Metrics Distribution Charts).

CS[22] in `example-8cores.btf.gz` (a context-switch stress task that migrates often):

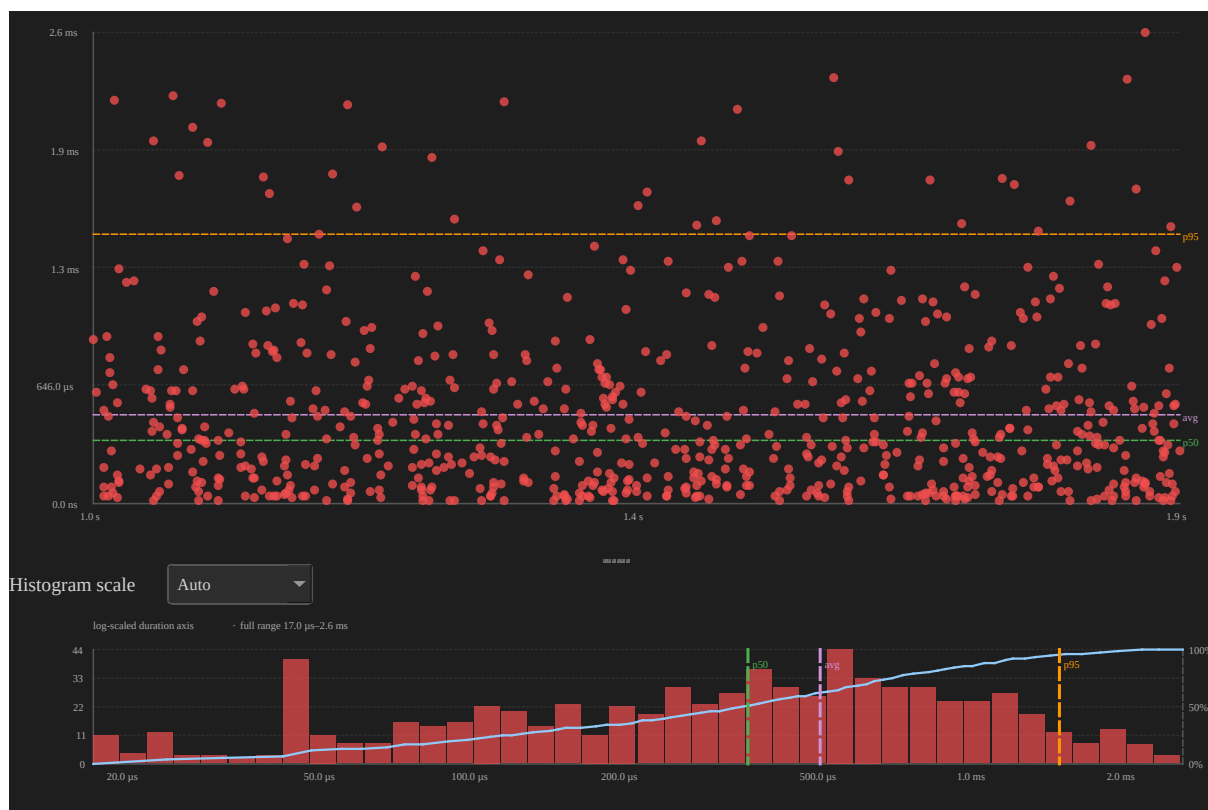


Figure 16: On-core dwell time distribution for CS[22] in example-8cores.btf.gz

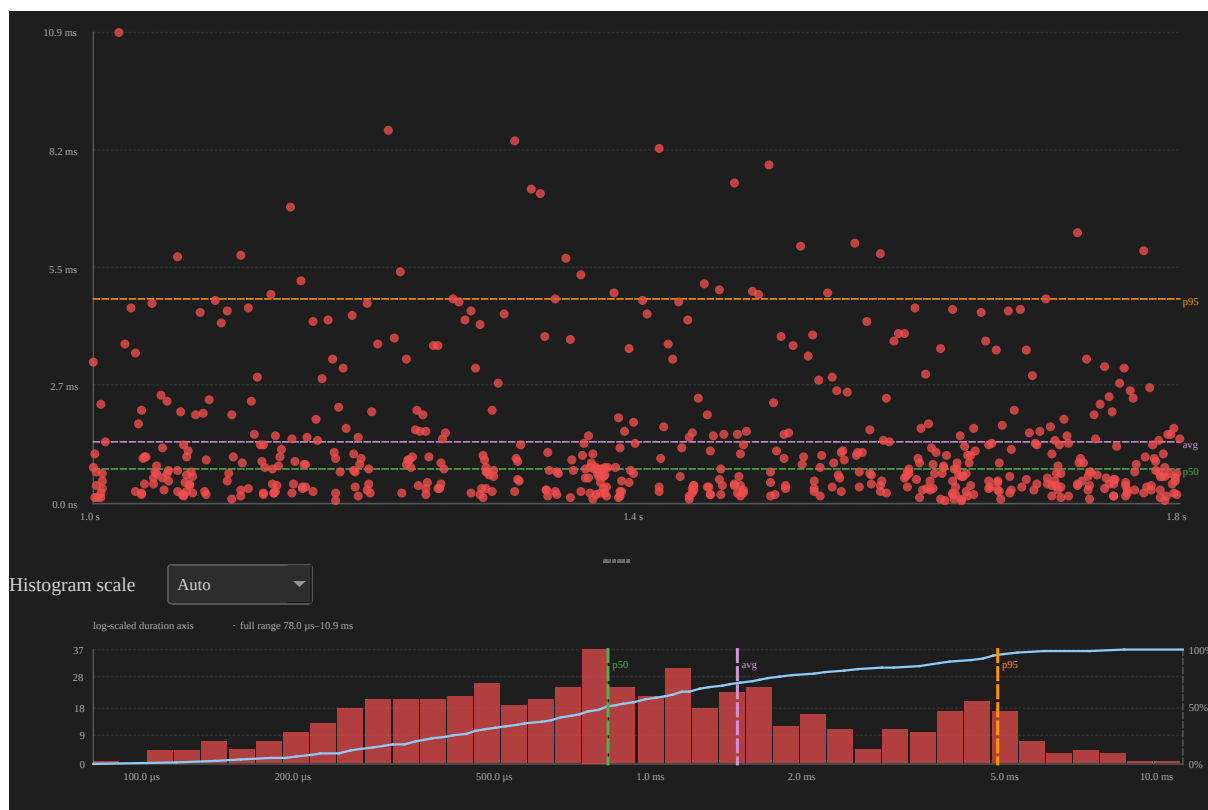


Figure 17: Time between migrations for CS[22] in example-8cores.btf.gz

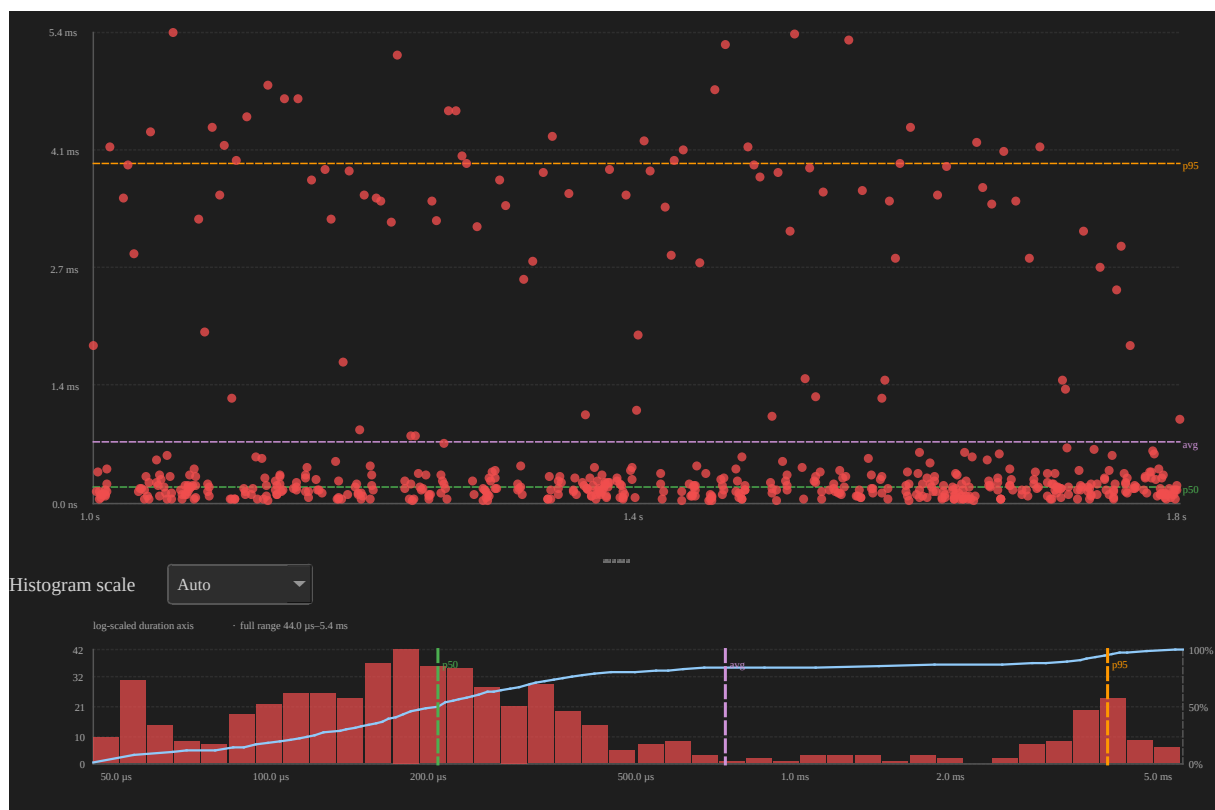


Figure 18: Post-migration gap distribution for CS[22] in example-8cores.btf.gz

Drag the resize handle below the table to show more or fewer rows.

Core-Pair Migration Summary Directed corridors (From → To) aggregated across all tasks. Complements per-task **Core Migrations** (which task thrashing?) with pair-level volume and lock-bounce share.

Column	Meaning
From / To	Source and destination cores
Count	Directed migrations in scope
Bounces	Subset that occurred while a mutex was held across cores
Bounce %	$100 \times \text{Bounces} / \text{Count}$

Column	Meaning
Avg Gap	Mean post-migration off-CPU gap for that corridor

Click a row to open a **distribution chart** with two tabs:

- **Gap** (default) — one point per directed migration with a positive gap: x = migration time, y = post-migration gap (the samples behind **Avg Gap**). Lock-bounce events are drawn in **orange**; others use the source-core colour.
- **Rate** — one point per consecutive migration on the *same* directed pair: x = migration time, y = time since the previous hop on this corridor. Tight vertical bands mean bursty corridor traffic.

Dialog footer:

- **Open Heatmap** — jump to the Migration Heatmap focused on that pair (prefers **Bounce Only** when Bounce % is elevated).
- **Open Chord** — open the chord diagram with the source core highlighted.

Core_5 → **Core_7** in `example-8cores.bt f.gz` (hottest corridor by Count):

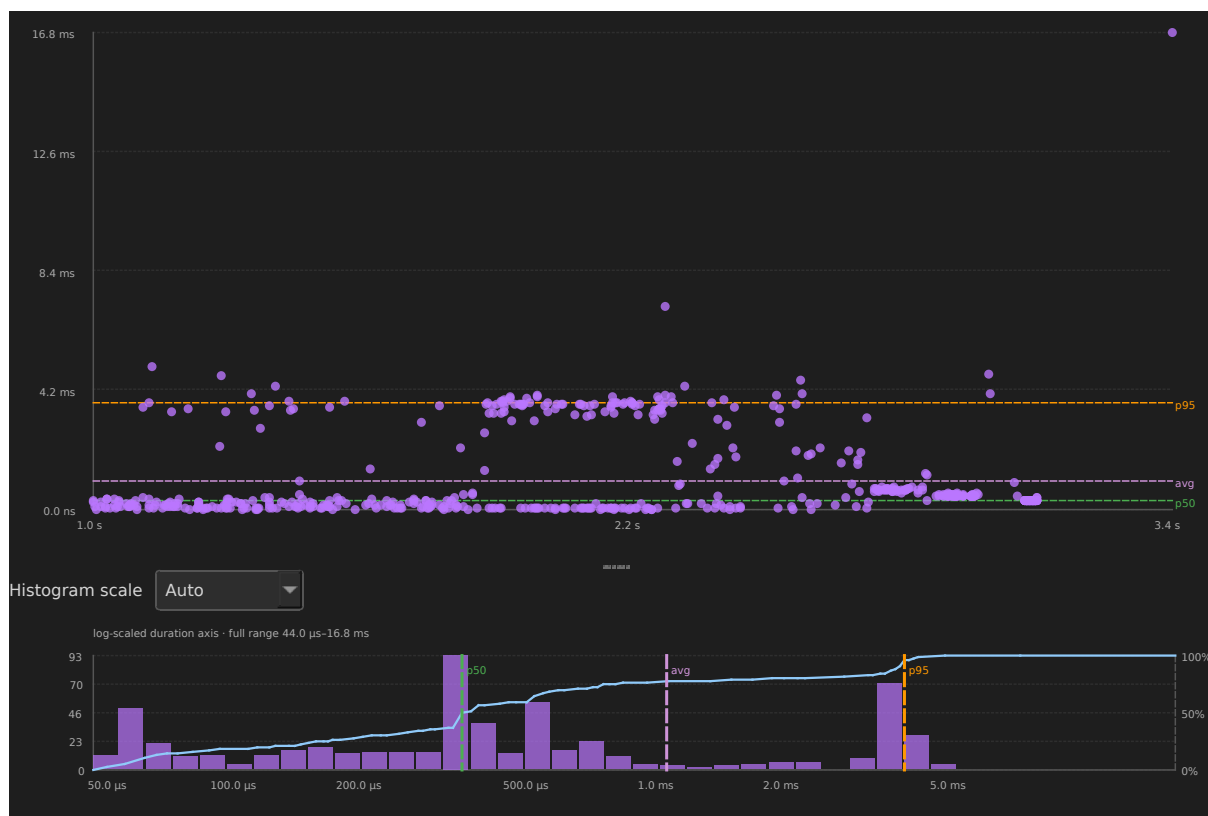


Figure 19: Post-migration gap for Core_5→Core_7

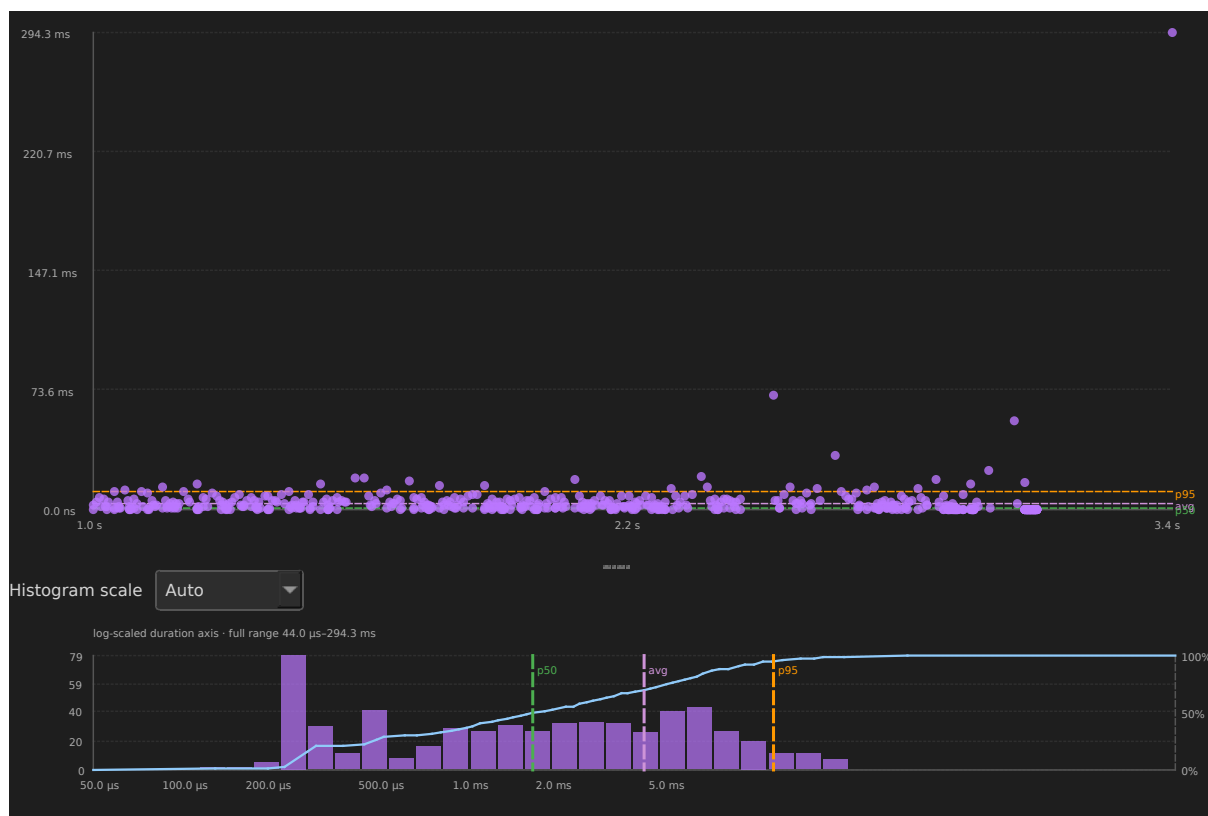


Figure 20: Time between pair migrations for Core_5→Core_7

Migration heatmap Visualise **when** migrations happen between core pairs — complementary to the per-task **Core Migrations** table. Traces with **more than 16 cores** use a **three-level** drill-down (core×core matrix → outgoing pairs × time bins → tasks). Smaller multi-core traces use a **two-level** flow (core-pair rows × time bins → tasks).

Bounce-only filter — the **Show: All Migrations / Show: Bounce Only** toggle button in the heatmap dialog restricts the visualisation to migrations that occurred while a mutex was held across two different cores (lock-bounce migrations, tracked via `lock_bounce_migration_ns`). Use this to identify which core pairs and time windows are dominated by cache-line bouncing rather than normal scheduling migrations.

Example (example-4cores.btf.gz) — screenshots exported with **Export SVG** from the heatmap dialog (full current drill level, no hover highlight):

Level 1 — core-pair overview (4 cores · 12 time bins across the trace):

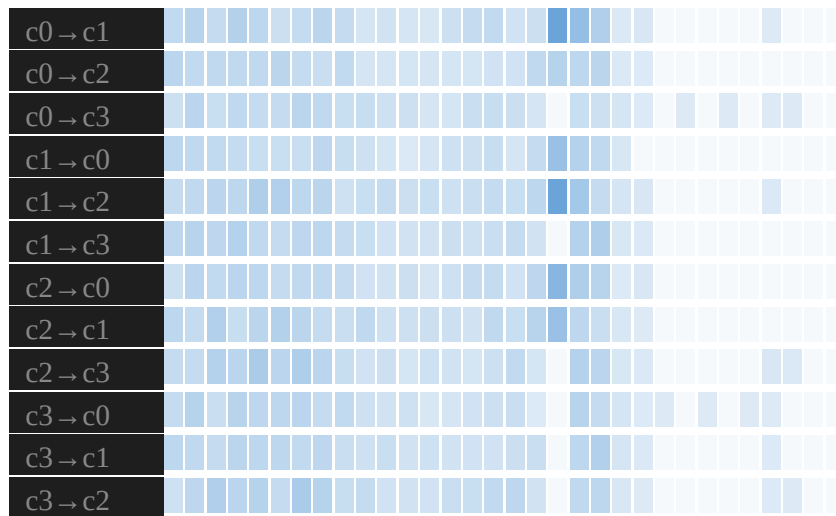


Figure 21: Migration heatmap Level 1: core-pair rows and time bins for example-8cores.btf.gz

Each row is a directed core pair ($c_0 \rightarrow c_1$, $c_0 \rightarrow c_2$, ...). Cell colour intensity is the migration count in that bin (darker blue = more events). Horizontal bands show **when** traffic occurred — e.g. repeated activity on $c_0 \rightarrow c_2$ and $c_1 \rightarrow c_0$ during the context-switch stress phases. Click a non-empty cell to drill into tasks for that pair and time window.

Level 2 — task grid (after clicking a cell on the pair overview):

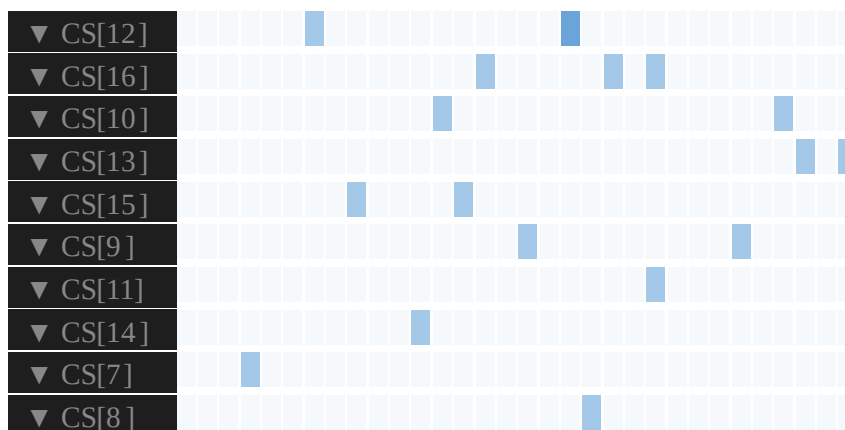


Figure 22: Migration heatmap Level 2: per-task sub-bins after drilling from example-8cores.btf.gz

Rows are tasks that migrated on the selected core pair within the chosen bin; columns are **12 sub-bins** spanning that bin's time window. Brighter cells mark sub-intervals where that task crossed cores most often. Each row label is prefixed with a directional indicator: ▲ = task primarily migrates in this direction (egress-dominant), ▼ = more migrations on the reverse path (ingress-dominant), ⇄ = roughly symmetric. Click a task cell to zoom the timeline, place **C1 / C2** at the sub-bin edges, switch to **Task View**, and filter to that task only.

Typical workflow (≤ 16 cores)

1. Open **Heatmap** from the toolbar (tracedata/example-8cores.btf.gz or example-16cores.btf.gz are good demos).
2. **Level 1** — click a hot cell on a core-pair row to open the task grid for that bin.
3. **Level 2** — click a task cell to zoom the timeline, place **C1 / C2**, switch to **Task View**, and show only that task.
4. **Show all tasks** — when done, clear the task filter and restore the timeline viewport, cursors, and highlights from before the heatmap was opened; the heatmap returns to the core-pair overview.

Typical workflow (> 16 cores)

1. **Level 1 — core×core matrix** — rows = source cores, columns = destination cores. Hover a row to highlight it; **click a row** to drill into outgoing migrations from that core.
2. **Level 2 — outgoing pairs** — one row per destination (c3→c1, c3→c2, ...), columns = 32 time bins. Hover highlights the row; click a cell to open the task grid for that pair and bin.
3. **Level 3 — tasks** — same as Level 2 above for smaller traces.
4. ← **Back** steps up one level; **Show all tasks** resets the timeline filter and returns to Level 1 (matrix or pair overview).

Open

Toolbar **Heatmap** (Desktop + Web). Enabled only when the trace has **2 or more cores** (single-core traces disable the button). Desktop: non-modal dialog (timeline stays interactive). Web: semi-transparent overlay.

Level 1 (≤ 16 cores)	One row per directed core pair (c0→c1, c1→c0, ...). Core names are shortened in the labels (e.g. Core_0 → c0).
Level 1 (> 16 cores)	Core×core matrix — row = source core, column = destination core. Hover highlights the row; click a row (not a single cell) to open outgoing pairs.
Level 2 (> 16 cores)	Outgoing pairs from the selected source core (c3→c1, c3→c2, ...) × 32 time bins. Row hover highlight; click a cell for tasks.
Level 1 columns	32 equal time bins across the scoped window. Cell colour intensity is the migration count in that bin (darker = more migrations). Hover for time range, count, and task names.
Level 2 rows	Task display names that migrated on the selected core pair in the selected bin.
Level 2 columns	32 sub-bins within the Level 1 cell's time window.
← Back	Step up one level (tasks → outgoing pairs → matrix, or tasks → pair overview on smaller traces).
Click cell	≤ 16 cores: Level 1 → tasks. > 16 cores: Level 2 → tasks. Details below.
Click row	> 16 cores only: matrix Level 1 → outgoing pairs for that source core.
All tasks (toolbar)	Shown next to Heatmap while a heatmap task filter is active. Same reset as Show all tasks below.

Show all tasks

Clears the task filter and **restores the timeline state captured when the heatmap was opened** (viewport, cursors, highlights, view mode). The heatmap returns to **Level 1**; heatmap time scope follows the restored cursors (full trace if fewer than two cursors were saved). Available from: toolbar **All tasks**, heatmap dialog **Show all tasks**, Legend **Clear** (Web: Marks page; Desktop: legend dock), or enabling **Migrated tasks only**. Loading a new trace or switching tabs also clears the filter (Desktop closes the heatmap dialog on tab switch).

Export PNG / SVG

Footer buttons export the **full current drill level** (all rows/columns, no hover highlight). Filenames: heatmap-`{level}`-`{timestamp}`.png or .svg (e.g. pairs, matrix, outgoing, tasks). Desktop: file save dialog + status bar confirmation. Web: direct download.

Scope

Full trace by default. With **2 or more cursors** placed, Level 1 uses the time window from **C1** through the last cursor (subtitle shows the range). Independent of the Statistics panel **Limit to cursor range** toggle. **Show all tasks** restores pre-heatmap cursors, so heatmap scope matches that saved window.

Empty state

No migrations in scope. (Level 1) or *No task migrations in this cell.* (Level 2) when no events fall in the current window.

What happens when you click a cell Only cells with at least one migration (count > 0) are clickable.

≤ 16 cores — core-pair overview (rows = $c_0 \rightarrow c_1, \dots$ · columns = 32 time bins)

1. **Click a cell** — Opens the task grid for that core pair and time bin.

> 16 cores — core×core matrix (rows/columns = cores)

1. **Hover a row** — Highlights the source core row.
2. **Click a row** — Opens outgoing pair rows ($c_N \rightarrow c_1, c_N \rightarrow c_2, \dots$) × 32 time bins.

Outgoing pairs / ≤ 16 cores Level 1 (rows = core pairs · columns = 32 time bins)

3. **Click a cell** — Opens the task grid for that pair and time bin.

Task detail (rows = task names · columns = 32 sub-bins)

4. **Click a cell** — The timeline zooms to that sub-bin, **C1** and **C2** are placed at the sub-bin edges, the view switches to **Task View**, and only that **task** row is shown.
5. **Another task cell** — Each click **replaces** the previous filter (last task wins).

Reset — Show all tasks

6. Clears the task filter, restores the viewport/cursors/highlights saved when the heatmap was opened, returns the heatmap to **Level 1** (matrix or pair overview), and shows every task row again.

Use the heatmap to spot bursts of cross-core traffic, drill into the contributing tasks, then jump to Task View for slice-level inspection. For aggregate per-task migration statistics (ping-pong, STI correlation, gap-after), use **Core Migrations** in the Statistics panel.

Migration chord diagram Visualise **how much** migration traffic flows between each pair of cores, as directional chords around a circle — one arc per core. Complementary to the **Migration heatmap** (which shows *when* migrations happen): the chord diagram gives a single, at-a-glance summary of *volume and direction* across the whole scoped window, with no drill-down levels.

- **Chords** — a curved band between two core arcs for each directed pair with at least one migration; chord width is proportional to migration count and its colour fades from the source core's colour to the destination core's colour. Pairs that migrate in both directions are drawn as two offset chords.
- **Hover a core arc** — highlights that core's arcs and fades all other chords/arcs; the fixed-height line below the diagram shows a breakdown (out / in totals, then each non-zero directed pair count for that core).
- **Bounce-only filter** — the same **Show: All Migrations / Show: Bounce Only** toggle as the heatmap, restricting chords to lock-bounce migrations (held across a core boundary).
- **Scope** — same as the heatmap: full trace by default, or **C1** through the last cursor when 2 or more cursors are placed.

Open

Toolbar **Chord** button (Desktop + Web). Enabled only when the trace has **2 or more cores**. Desktop: non-modal dialog (timeline stays interactive; closes if you switch to a different trace tab). Web: semi-transparent overlay (closes on tab switch).

Export PNG

Footer button exports the diagram exactly as rendered (no hover highlight). Filename: migration-chord-{timestamp}.png. Desktop: file save dialog. Web: direct download.

Export SVG

Footer button next to Export PNG; same content, vector format. Filename: migration-chord-{timestamp}.svg. Desktop: file save dialog. Web: direct download.

Empty state

No migrations in scope. when no migrations fall in the current window.

Example (example-8cores.btf.gz) — screenshot exported with **Export SVG** from the chord dialog (full trace scope, no hover highlight):

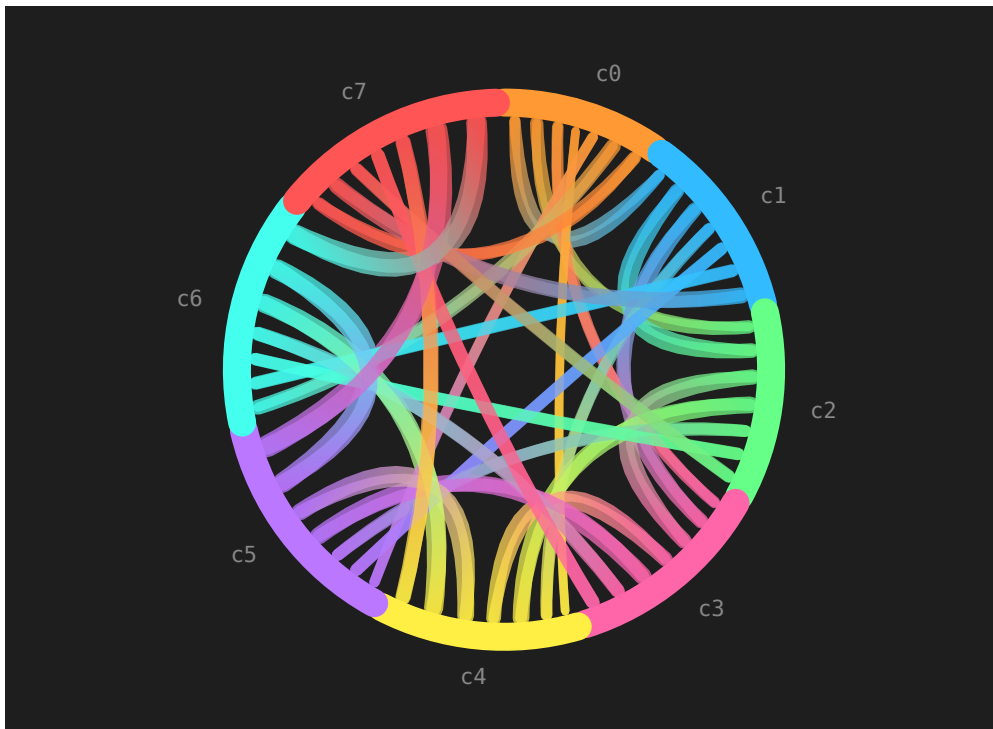


Figure 23: Migration chord diagram: directional core-to-core migration volume for example-8cores.btf.gz

Each arc around the circle is one core (c0...c7); a chord between two arcs is a directed migration flow, coloured as a gradient from its source core to its destination core, with width proportional to migration count. Thicker chords between adjacent-looking core pairs (e.g. c0↔c2, c1↔c6) stand out immediately as the busiest migration paths, without needing to drill through a heatmap grid first.

Trace Compare... Compare two traces you already have open side-by-side:

1. Open at least **two** .btf files (Desktop: **File** → **Open** adds a tab; Web: **Open** adds a tab in the bar under the toolbar).
2. In the **Statistics** panel footer, click **Trace Compare...** (enabled when two or more tabs are loaded).
3. Choose **Trace A** and **Trace B** from the dropdowns.
4. Optionally check **Limit to each tab's cursor range** to compare metrics within C1-Cn on each trace (requires 2+ cursors per tab).
5. Switch between **Summary**, **Top Tasks**, **Core Util**, **Core Migrations**, **Execution**, **Blocking**, **Inter-Arrival**, **Preemption**, and **Sync** tabs.

By default, compare views use the **full trace**. With the cursor-range checkbox enabled, each side uses that tab's own cursor window independently.

Summary — high-level diff:

Metric	Notes
Span	Total trace duration (or cursor-range width when scoped)
Tasks / Segments / STI events	Counts
Context switches	Total across all cores
Core gap avg / max	Idle time between consecutive slices on each core
Migrations (total) / Migrated tasks	Core-migration counts
Load Balance Score / σ	Same Gini-based score and util stddev as Statistics Core Utilisation
Tick health / mode / count / missed	Same Trace Health (TICK) summary as Statistics

Each row shows Trace A, Trace B, and **Δ** (signed difference).

Top Tasks — top 10 user tasks by CPU% from each trace, unioned by display name (Name[id]). Tasks present in only one trace show — for the missing side.

Core Util — per-core utilisation % (IDLE/TICK excluded), A vs B with Δ.

Execution / Blocking / Inter-Arrival — top tasks by sample count with Runs/Gaps, Avg, Max, and Δ (aligned with the Statistics metric tables).

Core Migrations — same columns as the Statistics panel, compared side-by-side:

Column	Meaning
Task	Display name (Name[id])
Migr A / B	Migration count in scope for that trace
Δ	Difference (A – B)
Rate A / B	Migration rate label (/s and /tick) per trace
Rate Δ	Signed difference of migrations per second of on-CPU time (A – B)
Dwell A / B	Average on-CPU slice duration per trace
Dwell Δ	Signed difference of average dwell time (A – B)
Ping A / B	Ping-pong count in each trace
Cores A / B	Distinct cores used in scope
Primary A / B	Primary core and % of on-CPU time

Preemption / Sync — victim totals and sync-object aggregates (holds, issues, lock-bounce / affinity violations, mutex/sem/queue counts).

Use this to compare builds, configurations, or runs of the same workload without merging traces manually.

Use case: tickful vs tickless (performance and context switches) Capture the **same workload** twice — once with a fixed tick (configUSE_TICKLESS_IDLE = 0) and once with FreeRTOS **tickless idle** (= 1) — then use **Trace Compare...** to quantify scheduler cost and application latency.

Sample pair: tracedata/tickless-8cores.zip (8-core demo; members tickful-8cores.btf + tickless-8cores.btf). Opening the zip in the GUI loads both as tabs; the headless CLI accepts the zip as a single compare input.

Capture (demo firmware)

```

# Fixed tick
make CORES=8 TICKLESS=0 run
cp tracedata/trace.btf tracedata/tickful-8cores.btf

# Tickless idle
make CORES=8 TICKLESS=1 run
cp tracedata/trace.btf tracedata/tickless-8cores.btf

# Optional: pack for GUI multi-tab open / CLI compare
zip -j tracedata/tickless-8cores.zip \
    tracedata/tickful-8cores.btf tracedata/tickless-8cores.btf

```

Keep STI **TICK** enabled on both builds and use the same suite / duration so Δ is meaningful.

Compare in the UI

1. Open the pair: `python builds/btf_viewer.py ../tracedata/tickless-8cores.zip` (two tabs), or open the two `.btf` files separately.
2. Optionally place matching cursor windows on the same busy (or idle) phase in each tab and enable **Limit to each tab's cursor range**.
3. Statistics footer → **Trace Compare...** → set Trace A / B labels (e.g. Tickful / Tickless).

What to read for performance and context switches

Compare tab / row	Why it matters
Summary → Context switches	Primary scheduler-activity cost between tick policies
Summary → Tick mode / Tick count / Tick health	Confirms config; tickful should favour lower CV when idle stretches dominate
Summary → Core gap avg/max, Load Balance Score / σ	Idle/busy structure and SMP balance
Summary → Migrations	Whether tick wake pattern changes cross-core bouncing
Execution (Max / p95)	Slice WCET and CPU-share shifts

Compare tab / row	Why it matters
Blocking (Max / p95)	Response-time impact under each policy
Preemption	Peer interference / tick-driven preemption differences
Top Tasks / Core Util	Who absorbs tick or wake-up overhead

CLI

```
# Zip with two .btf members (archive-root order → Trace A, Trace B)
python builds/btf_viewer.py compare ../tracedata/tickless-8cores.zip \
  -o /tmp/tick-policy.html --format html \
  --name-a Tickful --name-b Tickless
```

```
# Or two paths; optional shared busy/idle window (# timeScale units)
python builds/btf_viewer.py compare \
  ../tracedata/tickful-8cores.btf ../tracedata/tickless-8cores.btf \
  -o /tmp/tick-policy-busy.html \
  --name-a Tickful --name-b Tickless \
  --lo 1464000 --hi 1764000
```

Example Summary (tickless-8cores.zip, full-trace scope, A = Tickful, B = Tickless, $\Delta = A - B$):

Metric	Tickful	Tickless	Δ
Context switches	31,414	31,620	−206
Migrations (total)	19,018	18,440	+578
Tick count	2,561	2,611	−50
Load Balance Score	95 %	95 %	≈0
Span	2.421 s	2.444 s	−23 ms

On a **full** stress-suite capture (high core util), context-switch and tick counts can stay close — tick policy matters most in **idle-heavy** windows. Scope cursors (or --lo/- --hi) around an idle phase (e.g. demo test 11) when measuring power-oriented

tickless gains; keep a busy CS window when checking that latency budgets still hold.

Known limitation — SMP tickless idle: on the bundled tickless-8cores.zip sample, test 11's TICK STI still fires every ~1 ms in *both* builds (no widened gaps in the tickless capture). Root cause is upstream in FreeRTOS-Kernel/tasks.c's prvGetExpectedIdleTime(), which forces the expected idle time to 0 whenever more than one task is ready at idle priority. Under SMP, a *running* task stays in its ready list (only xTaskRunState changes), so with configNUMBER_OF_CORES = 8 all 8 per-core IDLE tasks are simultaneously "ready", and tickless idle never actually engages. This is a vanilla-kernel heuristic gap, not a capture or viewer bug — expect tickless vs. tickful tick counts to stay close on SMP builds until the kernel's idle-ready-list check is made SMP-aware.

Interpretation

Observation	Typical reading
Tickless: TICKLESS mode / higher CV; tickful: TICK , CV $\ll$ 5 % (often clearer in idle-scoped windows)	Configurations captured correctly
Context switches ↓ on tickless in idle-heavy windows	Expected — suppressed idle ticks reduce scheduler wake-ups
Context switches similar on a fully busy CS phase	Tick policy has little effect when cores never idle
Blocking / Execution Max worse on one side	Prefer that policy only if the Δ fits latency budgets
Migrations ↑ with one policy	Re-check affinity; tick wake pattern can change placement

Prefer **tickless** when idle power matters and scoped busy-window metrics stay within budget. Prefer **tickful** when Trace Health must stay GOOD or soft real-time slices cannot tolerate tick stretching. Longer walkthrough: WORKFLOWS.md §5.2.

Find → Migrations: lists migration boundary times; F3 / Shift+F3 jump between them (Desktop + Web).

Metrics Distribution Charts

In the **Statistics** panel, click any row in **Concurrent Core Active**, **Kernel Switch Overhead**, **Execution Time**, **Blocking Time**, **Dispatch / Scheduling Latency**, **Inter-Arrival**, **Core Migrations**, **Preemption Chain**, **Priority Inheritance**, **Interval Analysis**, or **Tag Analysis** to open a floating chart popup. In **Trace Health (TICK)**, use the **Tick Distribution...** button (bar-chart icon beside the mode badge when ≥ 2 ticks are in scope). **Core Migrations** popups additionally show in-dialog tabs (**Dwell / Rate / Gap**) and **Tag Analysis** popups show tabs (**Value / Interval**) to switch metrics without closing the chart.

- **Scatter plot** — each event plotted in trace time order so you can spot trends, bursts, or outliers.
- **Histogram** — adaptive bar chart of the value distribution:
 - **Auto scale** (default) picks **linear**, **p5-p95**, or **log duration** from the data spread so bars are not squeezed when min/max or outliers span a wide range.
 - **Histogram scale** dropdown (Desktop toolbar above histogram; Web above the chart): **Auto**, **Linear**, **p5-p95**, **Log duration**.
 - **Adaptive bin count** (Freedman-Diaconis, 12-80 bins) instead of a fixed 50-bin linear split.
 - **Overflow buckets** in p5-p95 mode — separate dimmed bars for values below p5 and above p95, with counts in the caption.
 - **Log-scaled counts** when one bin dominates (tall spike vs many small bars).
 - **CDF overlay** — cumulative distribution curve on the histogram (see CDF overlay below).
 - Dashed reference lines for **avg**, **p50**, and **p95**; caption shows the active scale and full min-max range.
- **Export PNG / SVG** — buttons in the chart footer save the current scatter + histogram.

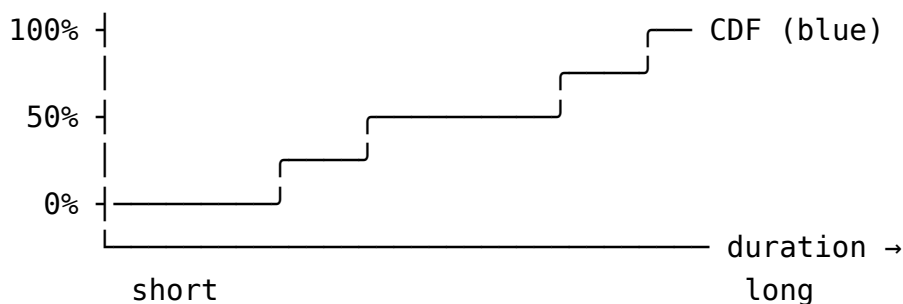
The popup can be dragged, resized, and closed independently of the main window. If the chart is open, it **updates live** when you move cursors or toggle cursor-range scope. Each open trace tab remembers its own chart when you switch tabs.

CDF overlay Every metrics histogram includes a **cumulative distribution function (CDF)** drawn as a **blue line** over the bars. It answers a different question than the histogram alone:

View	Question it answers
Histogram bars	<i>How many samples fall in each duration bucket?</i>
CDF curve	<i>What fraction of samples are at or below a given duration?</i>

The CDF is an **empirical** CDF (ECDF): for each sample in the active scope (full trace or cursor range), sorted by duration from shortest to longest, the curve plots **(duration → cumulative %)**. There is one step per sample; when several samples share the same duration, the curve rises vertically at that x position.

How to read the chart



- **Horizontal axis (bottom)** — duration (same scale as the histogram bars: linear, p5-p95, or log, depending on **Histogram scale**).
- **Left vertical axis** — sample **count** per bin (bar height).
- **Right vertical axis** — cumulative **percent** (0%, 50%, 100% labels on a dashed guide line).
- **Curve direction** — starts at the **bottom-left** (0% of samples below the shortest value) and rises toward the **top-right** (100% of samples included). A **steep** rise means many samples cluster at similar short durations; a **gradual** rise means values are spread out.

Relating the CDF to table columns and reference lines

The dashed vertical markers on the histogram are single-number summaries; the CDF shows the **full** cumulative picture:

Marker / table column	On the CDF
p50 (green)	Curve crosses 50% on the right axis at the median duration.
p95 (orange)	Curve crosses 95% — 95% of samples are shorter than this duration.
avg (purple)	Shown as a vertical line; the CDF does not pass through a fixed “avg %” because the mean is not a percentile.

Example readings (Execution Time for a task with 100 slices):

- At duration D where the CDF is at **30%**, roughly **30 slices** (30%) finished in D or less — useful for “how often does this task complete within my deadline?”
- If the curve is already above **90%** while still on the **left half** of the chart, most runs are short and the tail is light.
- If the curve stays below **50%** until far right, the distribution is **wide or skewed** — the histogram bars alone may look crowded; switch **Histogram scale** to **Log duration** or **p5-p95** and use the CDF to see where the bulk of the mass sits.

Histogram scale and the CDF

The CDF always reflects the **same samples** as the bars; only the **x mapping** changes with the scale mode:

Histogram scale	Effect on CDF
Auto / Linear	Duration mapped linearly from min to max.

Histogram scale	Effect on CDF
p5-p95	Main curve spans the p5-p95 window; outliers appear in dimmed underflow / overflow buckets at the edges, and the CDF steps into those buckets at the corresponding percentiles.
Log duration	Short and long durations are spread across the axis; the CDF is easier to read when bars would otherwise pile up on the left.

The caption above the histogram (e.g. log-scaled duration axis · full range 17 μs–975 μs) always shows the **true** min-max range even when the axis is compressed or clipped.

When to use the CDF

- **Deadline / budget checks** — estimate what % of activations meet a time limit without reading individual scatter points.
- **Compare spread** — two tasks with similar **p50** can have very different CDF shapes (tight cluster vs long tail).
- **Skewed data** — after switching away from a crowded linear histogram, use the CDF with **p50** / **p95** lines to see how much of the population sits below each marker.
- **Cursor-scoped analysis** — with **Limit to cursor range** enabled, the CDF recalculates for only the slices inside C1-Cn, same as the table and scatter plot.

The CDF is included in **Export PNG / SVG** from the plot dialog. It is not interactive (no click-to-jump); use the **scatter plot** above the histogram to jump to individual events.

Jump links: in Execution Time, Blocking Time, **Dispatch / Scheduling Latency**, and Inter-Arrival tables, click **Min** or **Max** (dotted underline) to jump to the slice at the shortest or longest value and add an **annotation** with a descriptive note. Click any **distribution-chart** point to jump to that event, add an annotation, and

switch to the **Marks** tab with the new annotation selected (segment start for task metrics; tick timestamp for **Tick Distribution**; switch/concurrency timestamp for those plots; zoom + highlight for **Priority Inheritance** episodes; interval start for **Interval Analysis**). In Preemption Chain, the annotation is placed at the **preemptor segment** start. In **Mutex / Semaphore**, click any **Pairing issues** row to zoom to the running task segment on that core, jump to the issue time, and add an annotation.

Example plots from `tracedata/example-4cores.btf.gz` (4-core SMP trace, 67 tasks) are in Statistics metric tables.

Export

Action	Desktop	Web
Annotated snapshot	Snapshot Editor / Save Image	Snapshot Editor
Copy viewport	Clipboard	Clipboard
SVG	Save SVG	Save SVG
Perfetto JSON	File → Export Perfetto...	Toolbar Perfetto
Statistics CSV/HTML	Statistics panel	Statistics panel
Compare CSV/HTML	Trace Compare dialog	Trace Compare dialog

Headless CLI (Desktop only)

Same engine as the GUI, suitable for CI. Use `QT_QPA_PLATFORM=offscreen` when no display is available.

Command	Purpose
<code>info</code>	Trace summary (<code>--json</code> optional)
<code>report</code>	Full statistics CSV/HTML

Command	Purpose
compare	Two-trace diff (two paths or one multi-BTF zip)
migrations	Migrations table as CSV
snapshot	PNG/SVG of timeline, heatmap, or a metric plot
perfetto	Chrome Trace JSON

```
python builds/btf_viewer.py info trace.btf
python builds/btf_viewer.py report trace.btf -o report.html --format html
python builds/btf_viewer.py compare before.btf after.btf -o diff.html
python builds/btf_viewer.py snapshot trace.btf -o view.png --view timeline
python builds/btf_viewer.py perfetto trace.btf -o trace.json
```

Run `python builds/btf_viewer.py <command> -h` for full options.

Settings

Open **Settings** from the toolbar or `Ctrl+,`.

Area	What you configure
Appearance	Dark/light theme, fonts, colorblind-safe palette
Display	Show/hide Legend, Statistics, Marks, Find, AI, STI, grid; hover highlight
Layout	Label width, row height, zoom 1:1 density, max cursors, time decimals, CPU/STI sizes
Analysis	CPU budget % and per-task deadline thresholds
AI	Enable, preset (Ollama / OpenAI / Gemini / Custom), base URL, model, API key, reply language

	Desktop	Web
Storage	btf_viewer.rc next to the viewer	Browser localStorage
Apply	Immediate	Save in the dialog (live preview while open; Cancel reverts)

Sessions also restore open tabs, viewport, cursors, and marks (Desktop: settings file; Web: local storage plus cached traces). Use **Reset to defaults** in Settings to clear custom preferences.

Keyboard & mouse

Shortcuts marked **(W)** are Web-only. Others work on Desktop and Web. On Web, press ? for an on-screen cheat sheet.

File & tabs

Key	Action
Ctrl+O	Open file (new tab)
Ctrl+W	Close tab
Ctrl+Tab / Ctrl+Shift+Tab	Next / previous tab
Ctrl+Q	Quit (Desktop)

View & zoom

Key	Action
Ctrl++ / Ctrl+-	Zoom in / out
Ctrl+0 / F	Fit to window
Ctrl+R	Zoom to cursor range
Ctrl+,	Settings
G / I / D	Grid / STI / theme
Ctrl+G	Jump to time
Ctrl+Home / Ctrl+End	Trace start / end

Edit & marks

Key	Action
Ctrl+Z / Ctrl+Y	Undo / redo
C / Shift+C	Place / clear cursors
Ctrl+B / Shift+B	Bookmark / clear bookmarks
Ctrl+Shift+B / A	Annotation
Ctrl+F / F3 / Shift+F3	Find / next / previous

Export

Key	Action
Ctrl+S / S (W)	Snapshot editor
Ctrl+Shift+C	Copy viewport
Ctrl+Shift+S	Save SVG
Ctrl+Shift+E	Export Perfetto

Mouse

Action	Effect
Wheel / Ctrl+wheel	Pan / zoom
Left-drag background	Pan
Middle-drag	Zoom to selection
Click timeline	Place cursor
Drag cursor or mark	Move
Right-click	Context menu

Further reading

Document	Audience
WORKFLOWS.md	Analysis playbooks and AI ask order
btv-viewer-slides.md	Presentation overview

Developer notes

Day-to-day users can ignore this section.

Task	Command
Rebuild Desktop + Web	<code>make -C BTFViewer</code>
Desktop package only	<code>make -C BTFViewer bundle → builds/btf_viewer.py</code>
Web only	<code>make -C BTFViewer web → builds/btf_viewer.html</code>

Task	Command
Tests	<code>make -C BTFViewer test</code>
Dev run (Desktop)	<code>python -m btf_viewer_pkg [trace.btf] from BTFViewer/</code>

Edit sources under `btf_viewer_pkg/` and `web/`; commit regenerated files under `builds/` with your changes. Synthetic traces: `scripts/gen_trace.py --help`. BTF field reference and format details live with the FreeRTOS-BTF instrumentation docs in this repository—not in this user guide.

Contributors

Thanks to everyone who has contributed to this project.

Contributor	Contribution
DiogoRoseira	CPU Load Graph and metrics distribution charts
